

Programación de GPIO para Sistemas de Control Digital con STM32

1- Dr. Lic. Roberto Federico FARFÁN

farfan.roberto.f@gmail.com

2- Esp. Ing. Luis Mariano CAMPOS

1,2-Profesor de Introducción a los Sistemas Embebidos - **Facultad de Ingeniería – UBA**

1-Profesor de Electrónica, Electrónica Analógica y Digital - **Facultad de Ingeniería - U.N.Sa**

2- **CONICET**

Resumen de la clase de hoy:

- 1.1 Temas para la semana 1.
- 1.2 Libro de referencia.
- 1.3 GPIO (Entradas y Salidas de Propósito General).
- 1.4 Reloj. Cube MX.
- 1.5 Primera Prueba.
- 1.6 Problemas.

Temas para la semana 1

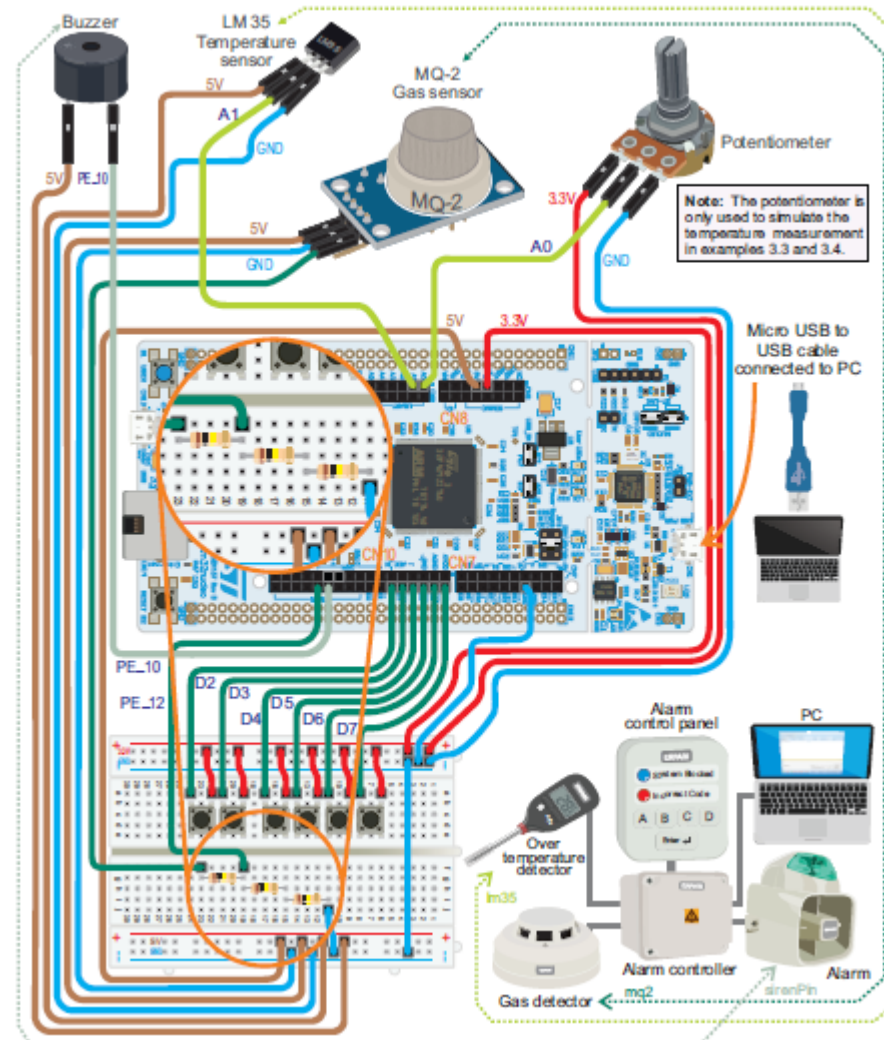


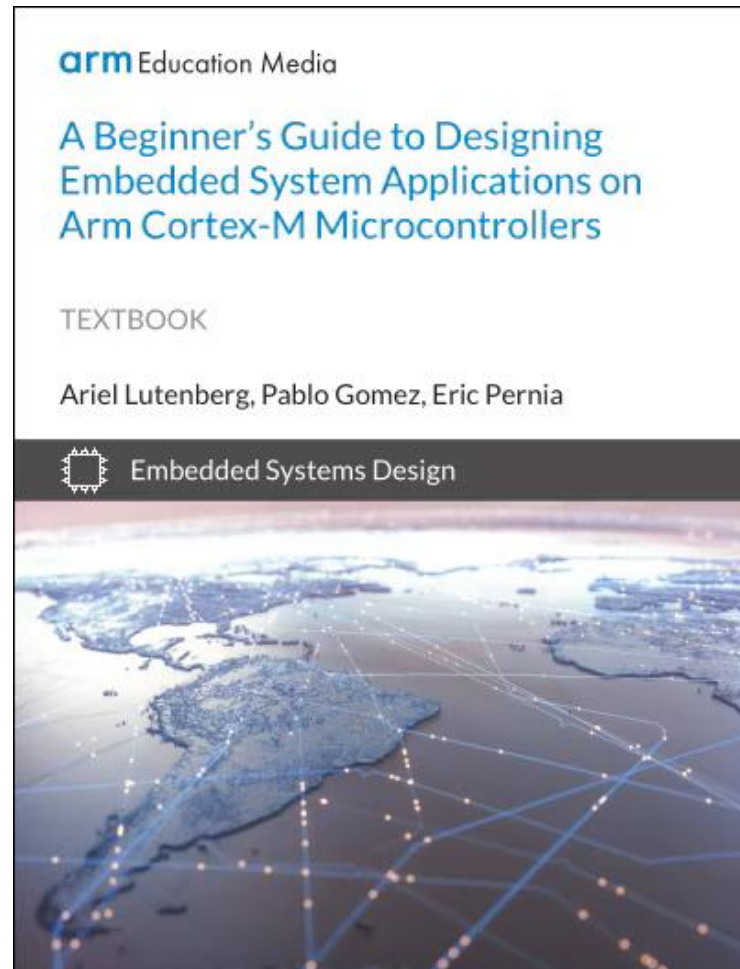
Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

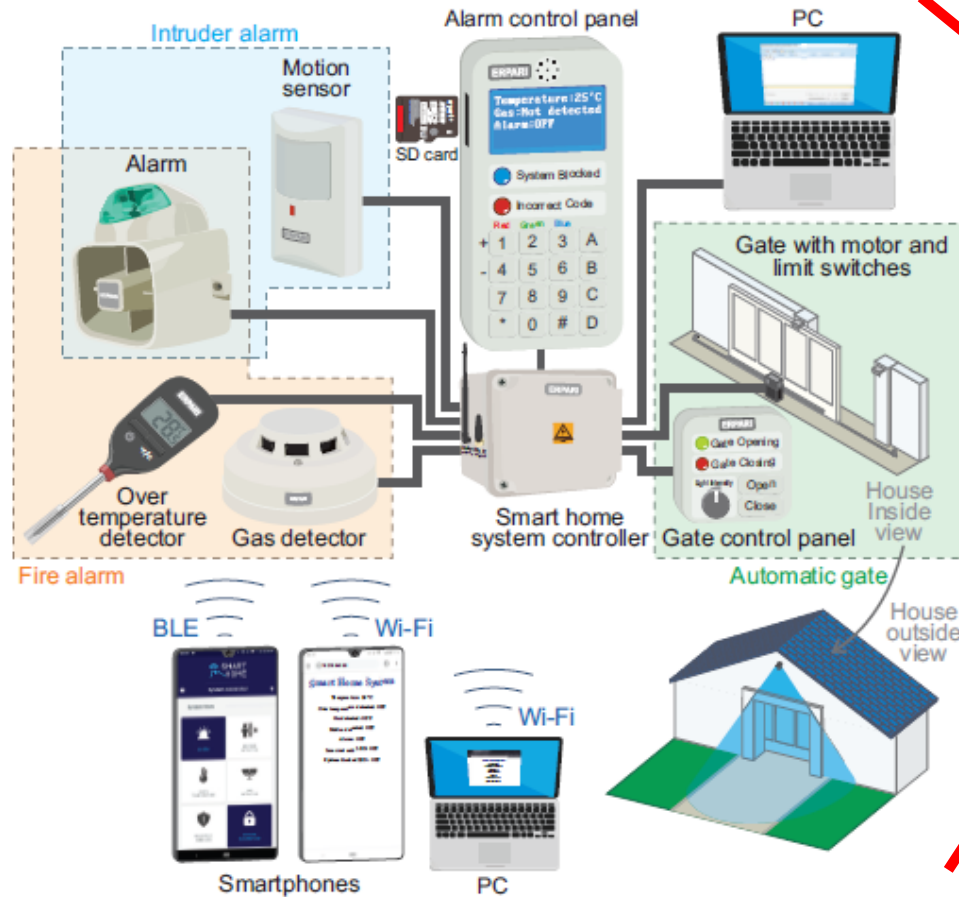
1.1 Temas para la semana 1.

Semana	Temas de teoría/práctica	Fecha entrega informe TP	Bibliografía básica
1	Contenido de las clases de la semana 1. Teoría: Definición de sistemas embebidos. Procesadores ARM Cortex-M: arquitectura y bloques internos. Concepto de álgebra booleana y entradas/salidas digitales. Entorno STM32CubeIDE. Práctica: Programación básica de GPIO (Entradas/Salidas). Estructura del proyecto de la materia.	Será informada por el docente a cargo.	1, 2, 3 y 4.
2	Contenido de las clases de la semana 2. Teoría: Protocolos de comunicación entre circuitos integrados. Comunicación Serie. Node-RED. Instalación y características de la programación de flujos. Práctica: Configuración del puerto serie. Envío y recepción de datos. Configuración del nodo de comunicación serie en Nodo-RED. Conexión Node-RED y placa NUCLEO-F446RE.	Será informada por el docente a cargo.	1, 2, 3 y 4.

Libro de Referencia



1.2 Libro de referencia.



arm Education Media

A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers

TEXTBOOK

Ariel Lutenberg, Pablo Gomez, Eric Pernia



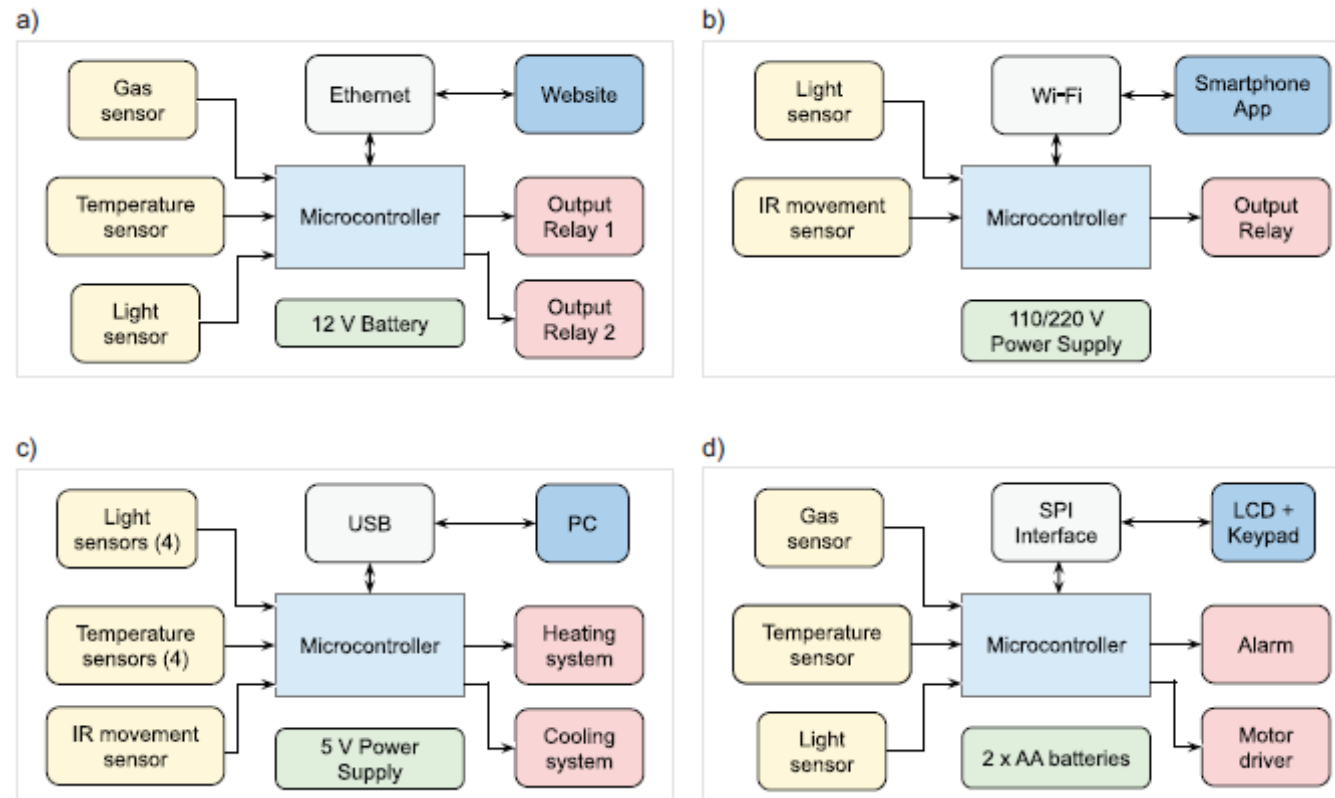
Embedded Systems Design



<https://www.arm.com/resources/education/books/designing-embedded-systems>

1.2 Libro de referencia.

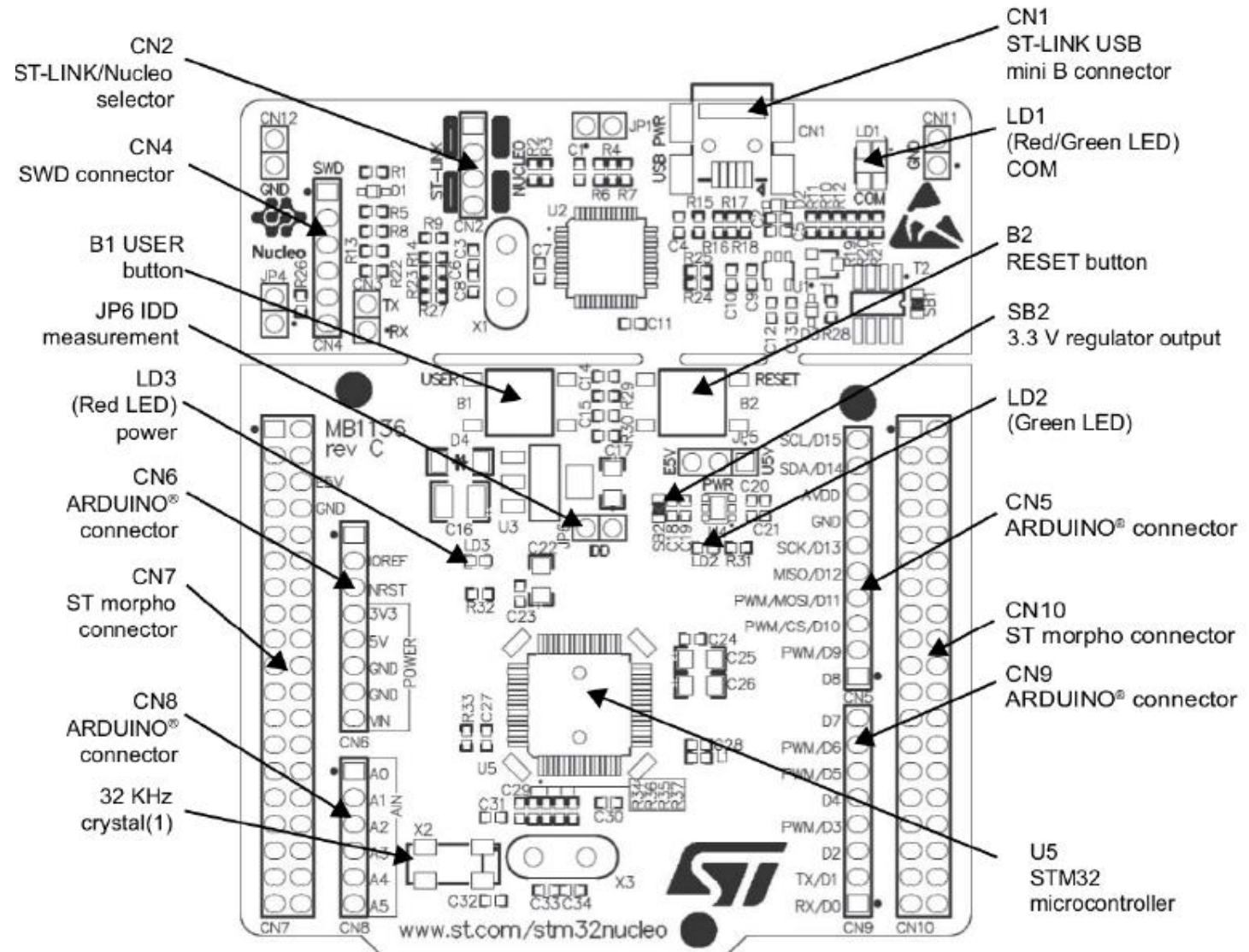
- Se implementa un proyecto de sistema de hogar inteligente.



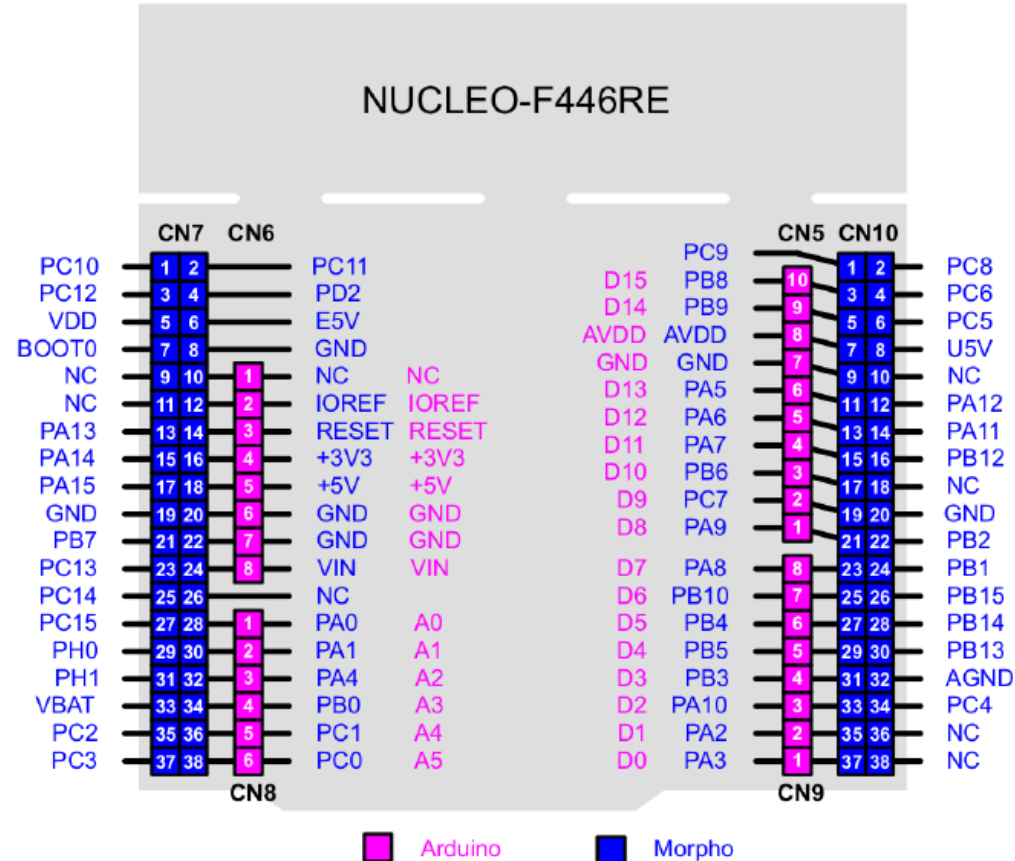
- Se presentan cuatro implementaciones.

1.2.1 Placa Núcleo-F446RE.

Componentes de la Núcleo.

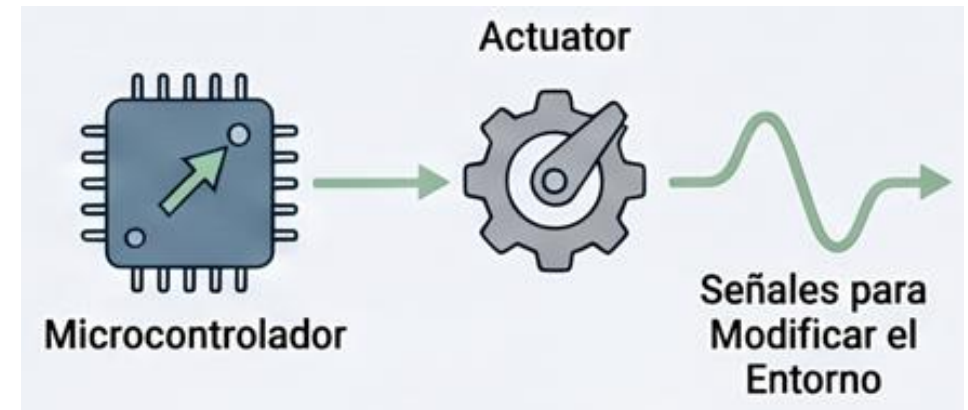


GPIO, Entradas y Salidas de Propósito General.



1.3 GPIO (Entradas y Salidas de Propósito General)

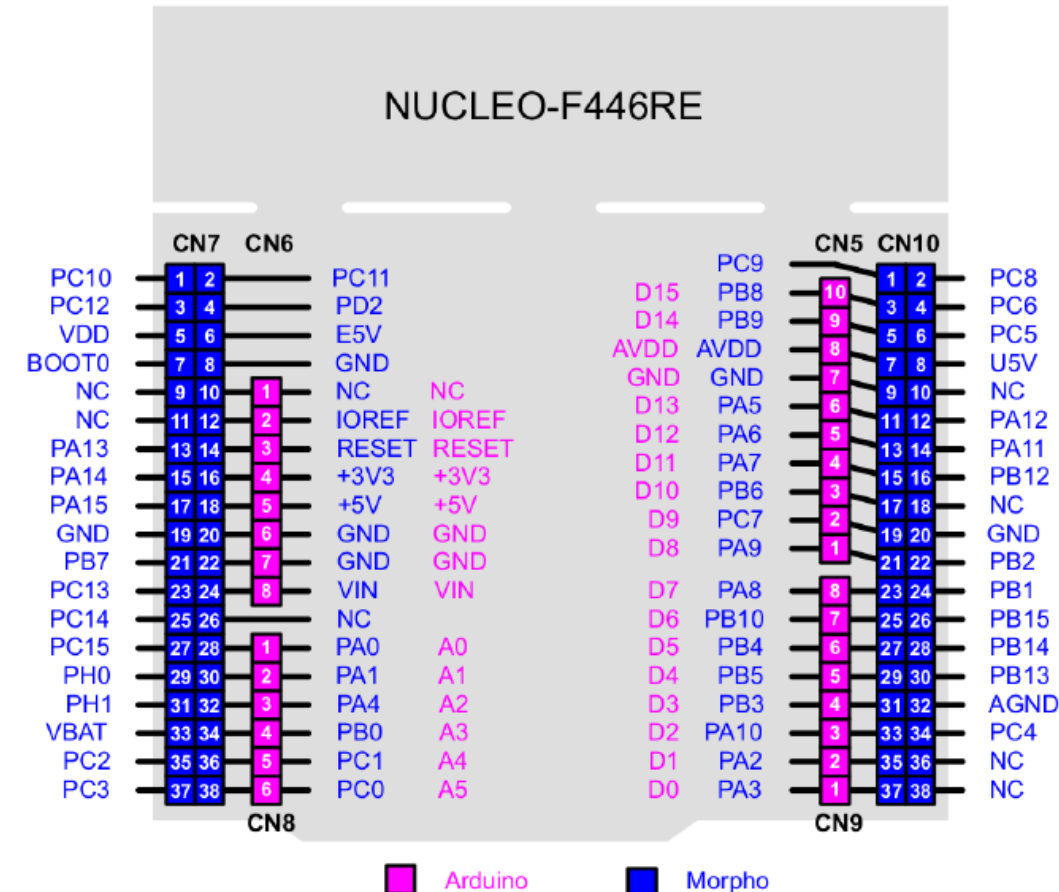
- Periféricos que permiten al μ C interactuar con su entorno físico, al capturar información o actuar sobre él.
- Las GPIO se organizan en puertos designados por letras (GPIOA o GPIOB).
- Los Puertos suelen tener 16 pines numerados del 0 al 15.



1.3 GPIO (Entradas y Salidas de Propósito General)

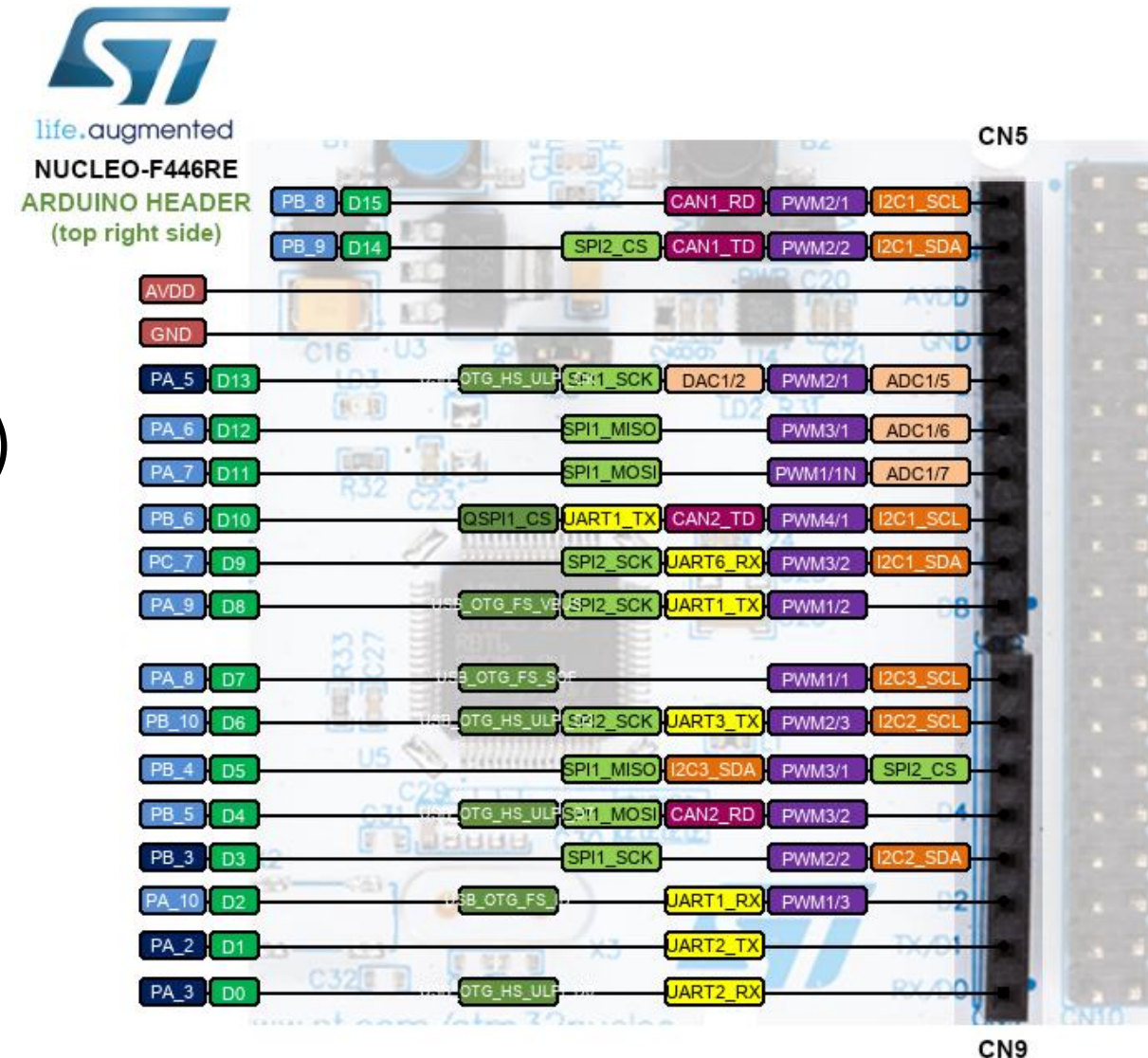
- El STM32F446 tiene “8 puertos”.

Encapsulado	Pines de E/S (GPIO)	Puertos Disponibles
LQFP64 (Nuestra placa)	50	PA, PB, (partes de) PC, PD, PH
LQFP100	81	PA a PE, (partes de) PH
LQFP144	114	PA a PH completos



1.3 GPIO (Entradas y Salidas de Propósito General)

- Los pines están multiplexados, pueden actuar como un GPIO estándar o desempeñar funciones como: comunicación (UART, SPI, I2C) o salidas de temporizadores (PWM), entre otras.



1.3.1 Modos de Configuración.

- Cada pin puede configurarse en los siguientes modos:
- Entrada: Puede ser flotante, con resistencia de Pull-Up (conectada a VDD) o Pull-Down (conectada a tierra), para evitar estados lógicos indeterminados.
- Salida: Puede configurarse como Push-Pull (capaz de suministrar corriente en alto y bajo) o Open-Drain (drenador abierto, requiere resistencia externa para el estado alto).

¿Que es una una Pull-up o Pull-down?

1.3.1 Modos de Configuración.

¿Que es una una Pull-up o Pull-down?

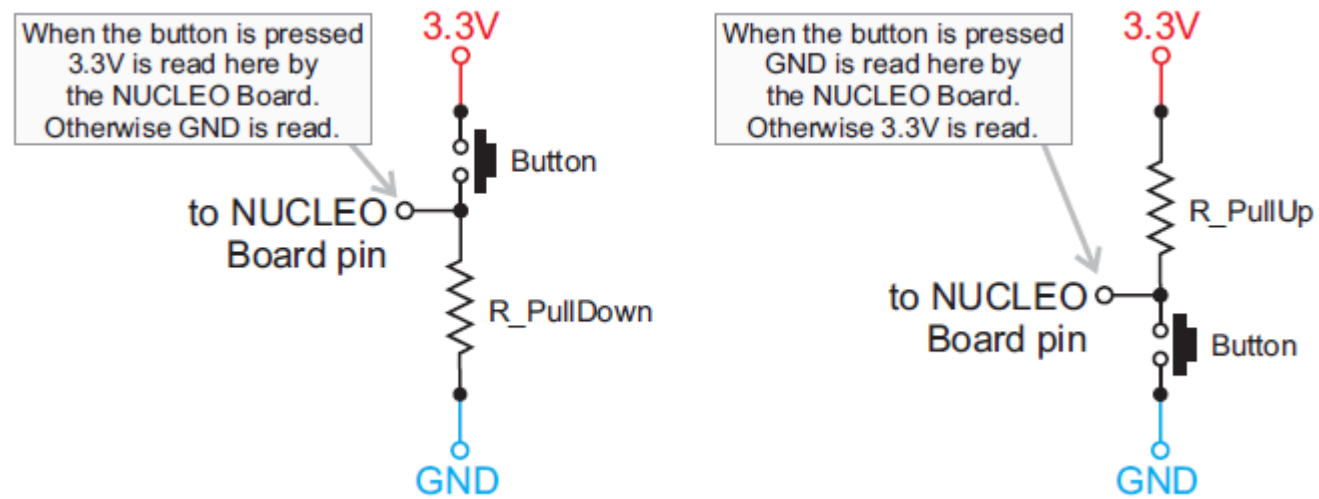
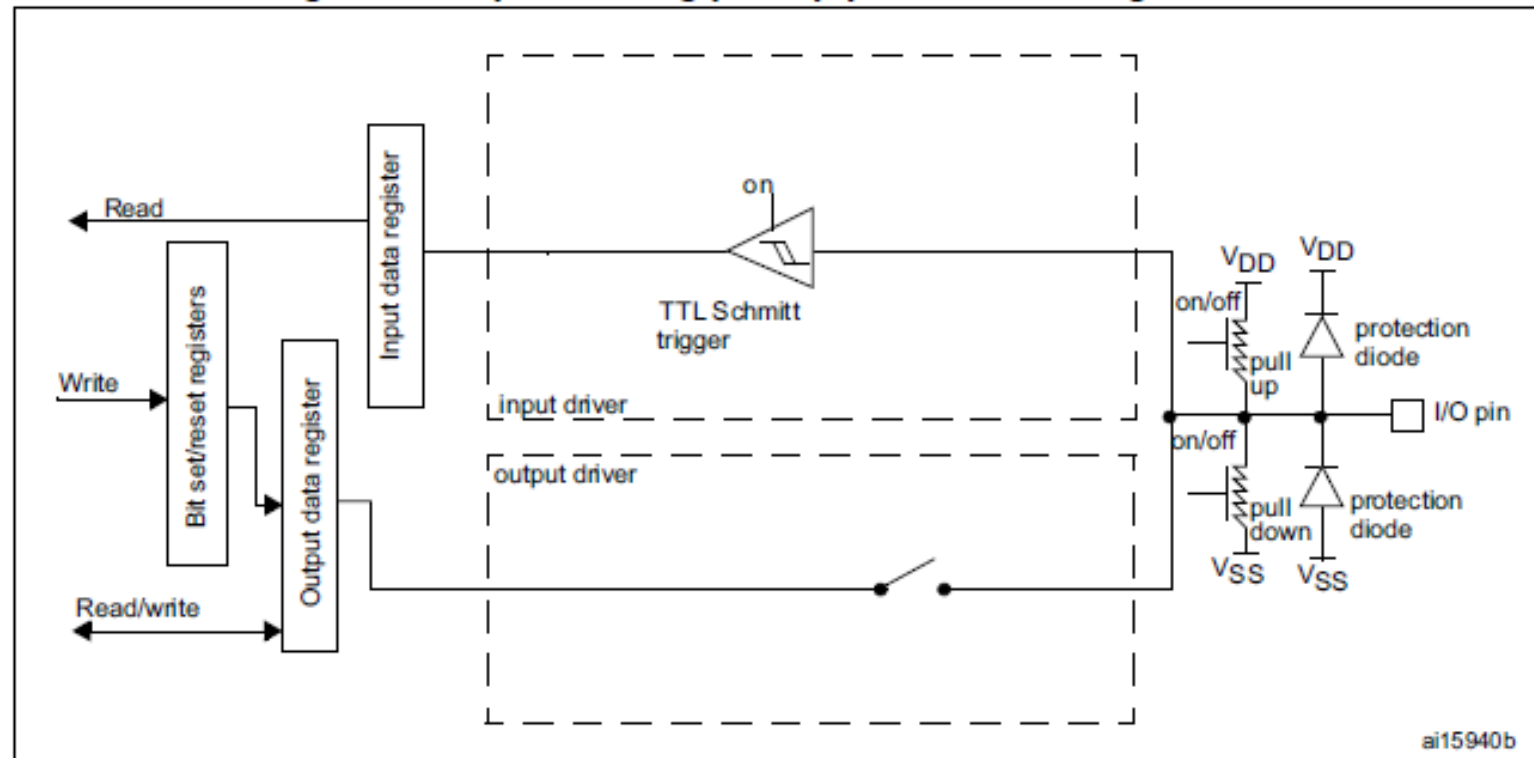


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers.
<https://www.arm.com/resources/education/books/designing-embedded-systems>

1.3.1 Modos de Configuración.

- Pull-Up y Pull-Down dentro del micro.

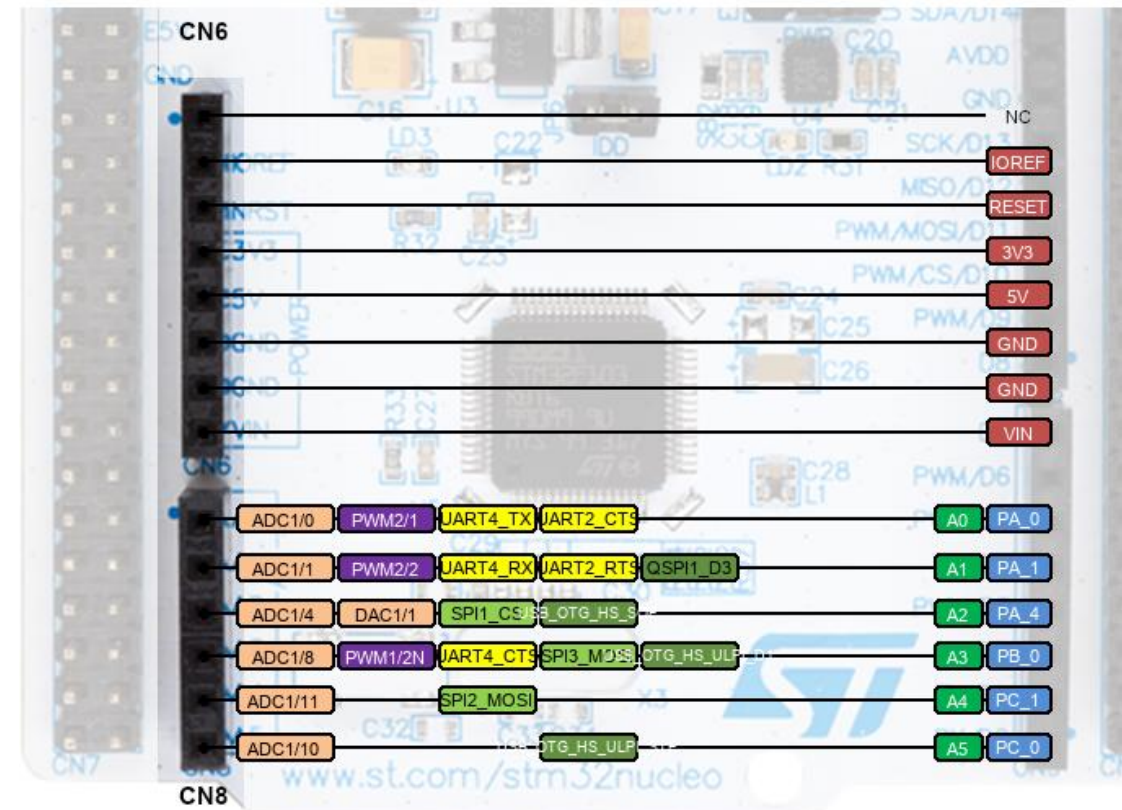
Figure 20. Input floating/pull up/pull down configurations



1.3.1 Modos de Configuración.

- Analógica: El pin puede utilizarse como un conversores, ADC (analógico-digital) o DAC (digital-analógico).
- Función Alternativa: En el pin se puede generar una señal PWM.

Figura obtenida de <https://os.mbed.com/platforms/ST-Nucleo-F446RE/>



1.3.2 Características Eléctricas y de Velocidad

- Corriente: Los GPIO permiten el manejo de corrientes de hasta 25 mA por pin, pero con un límite total para todo el chip de 120 mA.
- Para consumos mayores se requieren utilizan transistores o relés.
- Opera usualmente a 3.3V, pero existen pines específicos denominados (FT) que toleran 5V sin dañarse.
- Es posible configurar la velocidad de conmutación, en algunas situaciones hasta 90 MHz.

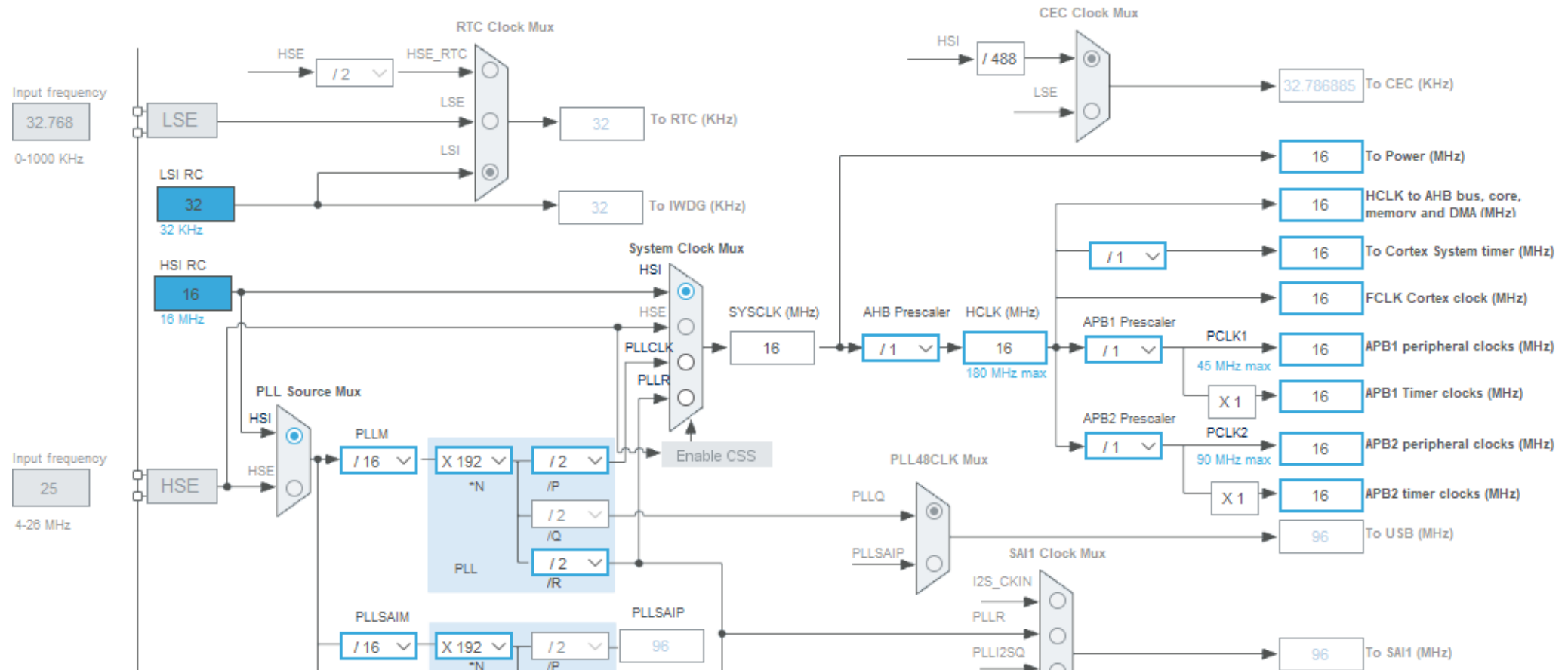
1.3.3 Gestión mediante Registros (Bare Metal)

Permite programar los GPIO a bajo nivel por medio de registros específicos de 32 bits:

- Configuración: El Registro MODER define el modo de trabajo, el PUPDR configura pull-up/down y el AFRL/AFRH asigna funciones.
- Datos: El registro IDR se usa para leer el estado de las entradas y el ODR para escribir el estado de las salidas.

Esta forma de configurar periféricos no vamos a utilizarla, en este curso utilizamos las herramientas HAL.

Cube MX. Reloj.



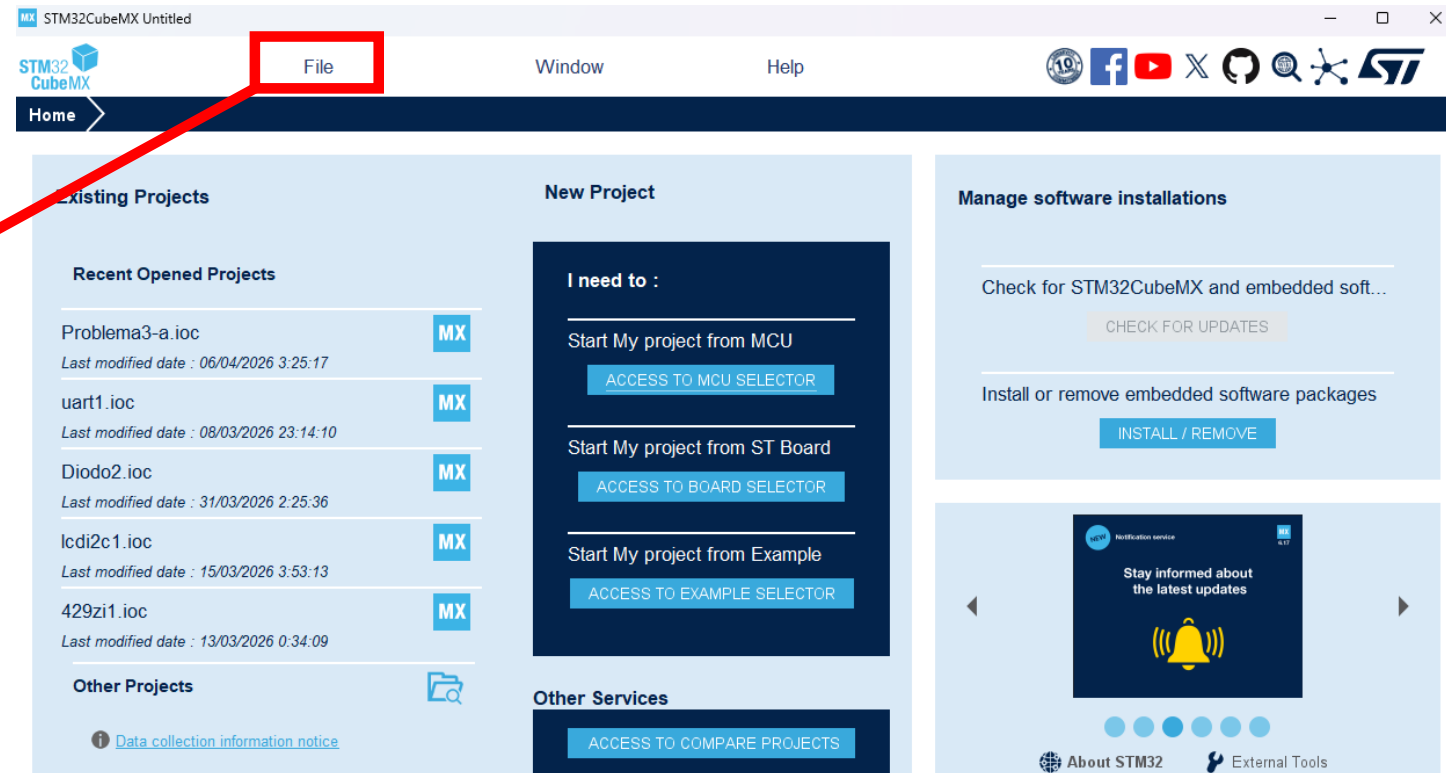
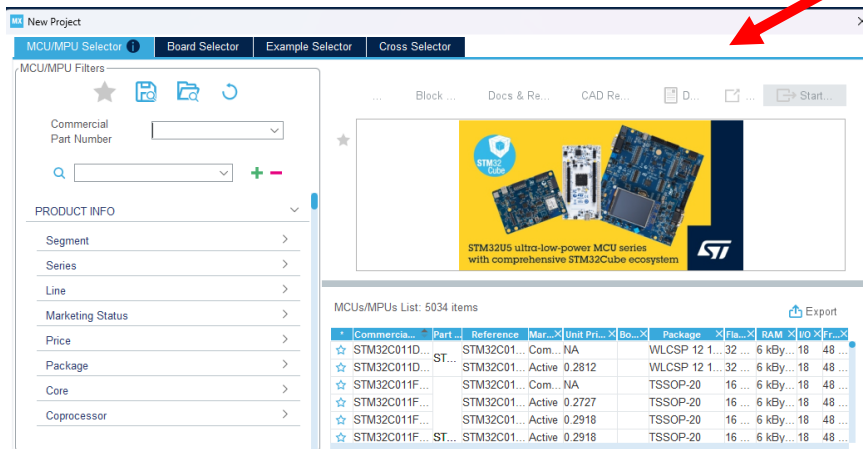
1.4 Cube MX. Reloj.

En los siguientes enlaces encontramos documentación y software:

- Documentación de la placa de desarrollo:
https://www.st.com/content/st_com/en.html
- Aquí encontramos el Cube MX:
<https://www.st.com/en/development-tools/stm32cubemx.html>
- Aquí encontramos el Cube IDE:
<https://www.st.com/en/development-tools/stm32cubeide.html>

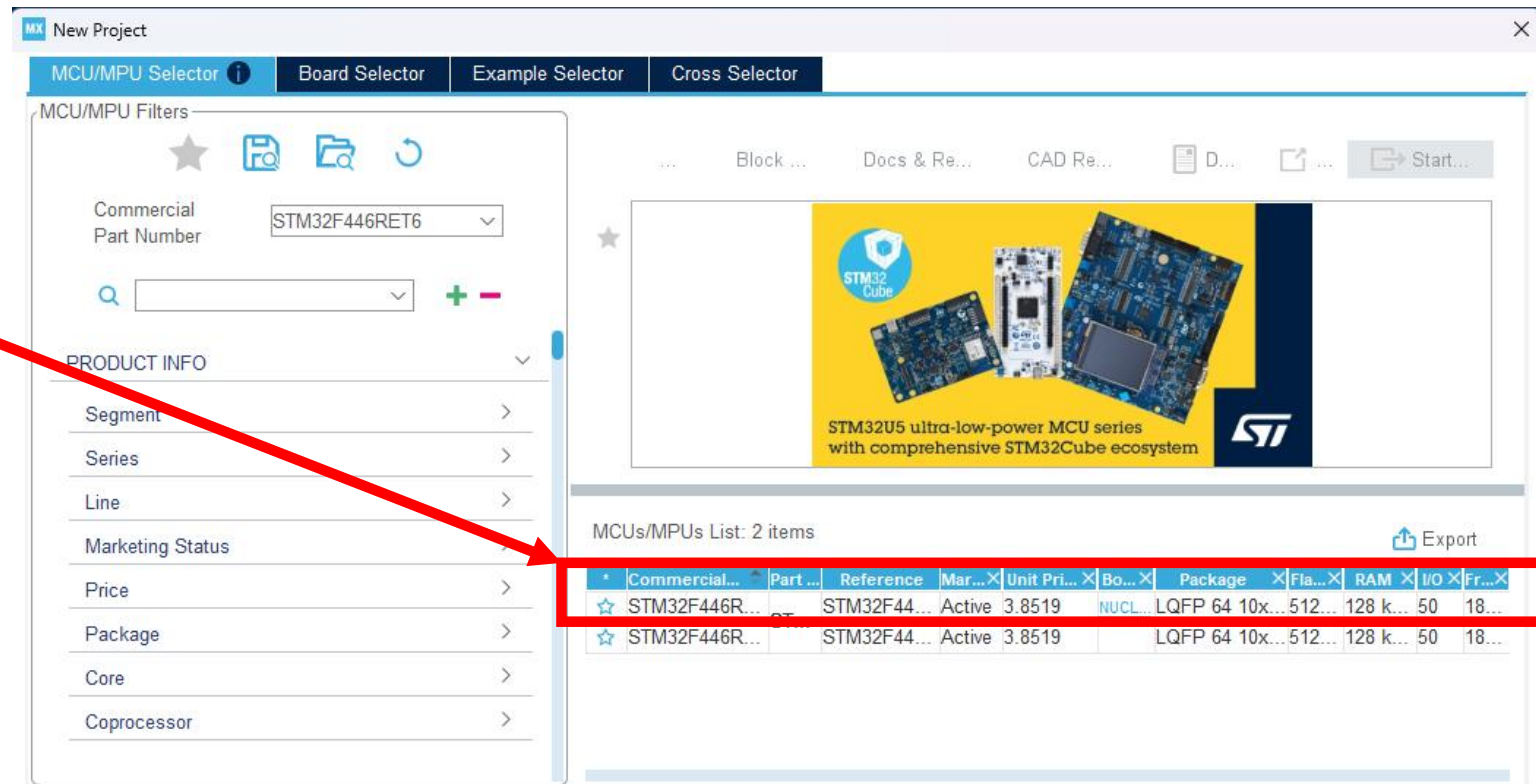
1.4.1 STM32CubeMX.

- Es un entorno de desarrollo que facilita la configuración de los periféricos, el cual genera código inicial de la configuración.
- 1- Abre STM32CubeMX y se busca a:
File -> New Project.



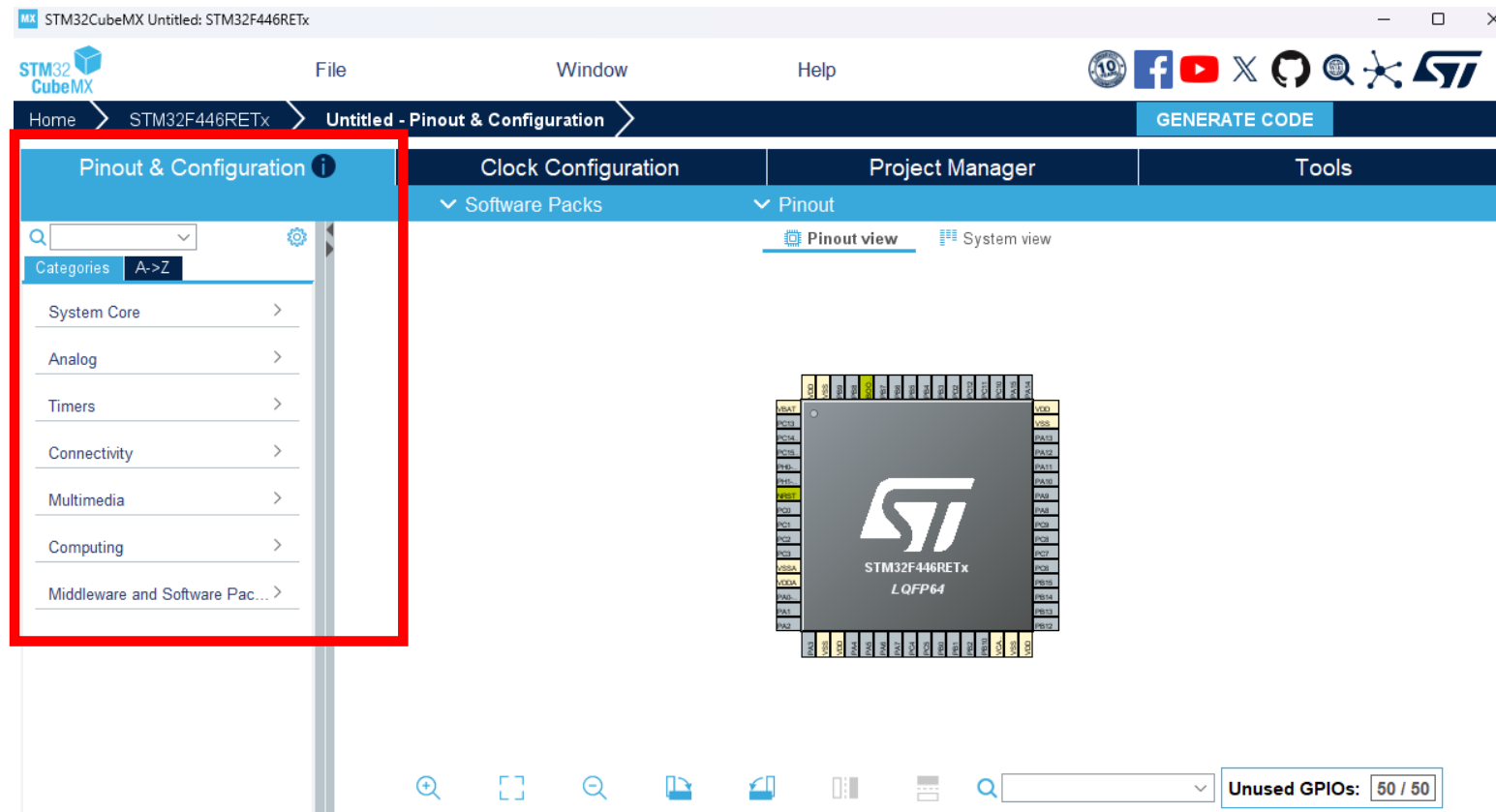
1.4.1 STM32CubeMX.

- En la ventana **MCU/MPU Selector**, en el buscador **Commercial Part Number** buscar STM32F446RE.
- Seleccionar la primera opción.



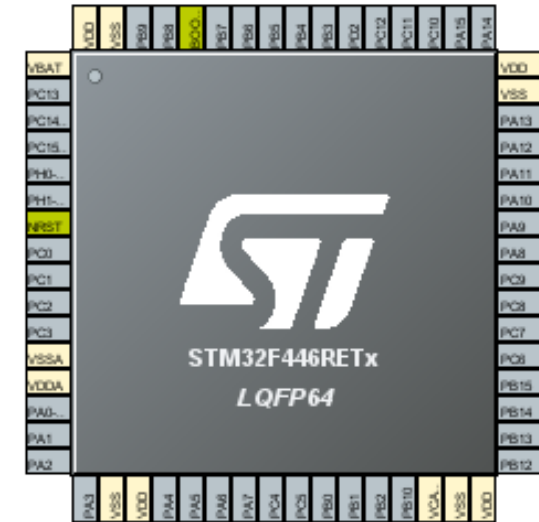
1.4.1 STM32CubeMX.

- Ventaja: Se pueden configurar el **LED** de la placa (PA5), el **Botón Azul** (PC13), la consola serie (USART2), entre otros.



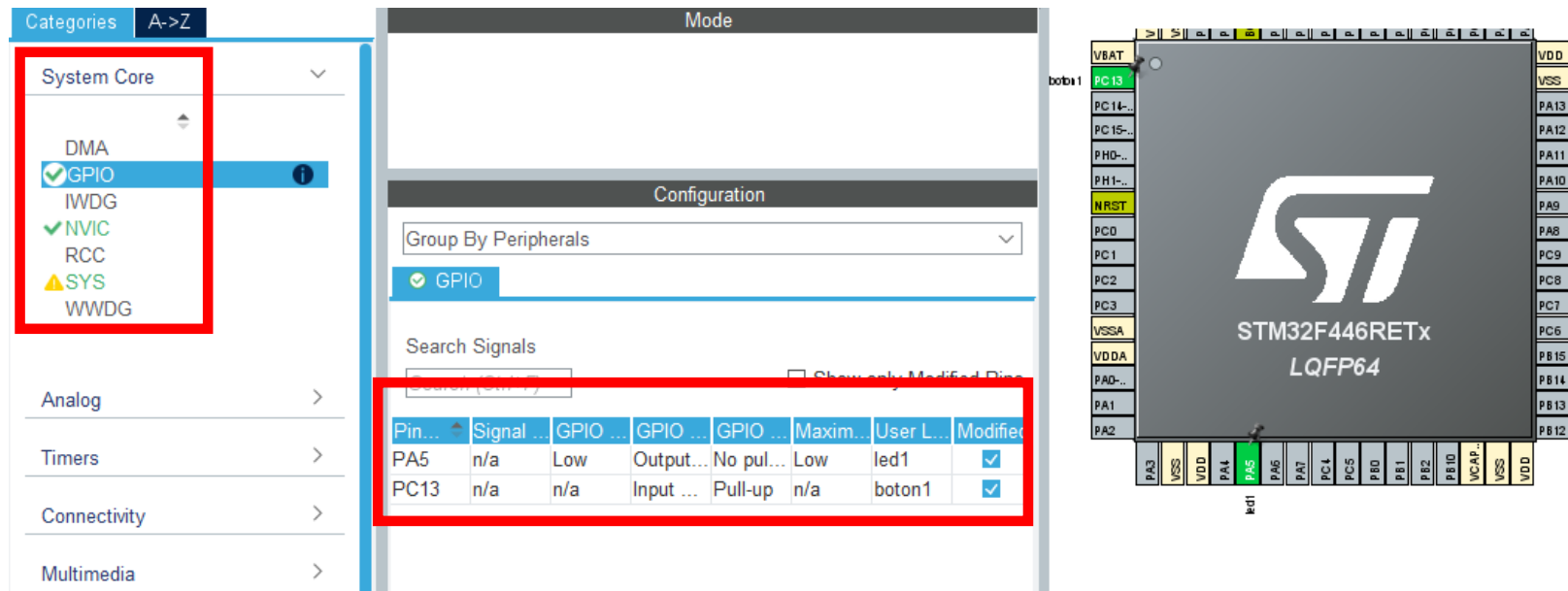
1.4.1 STM32CubeMX.

- Configuración de Pines:
- Entradas: Se realiza un clic izquierdo en los pines que elijas y seleccionar GPIO_Input.
- Salidas: Si se conecta un LED externo, se realiza un clic en otro pin y selecciona GPIO_Output.



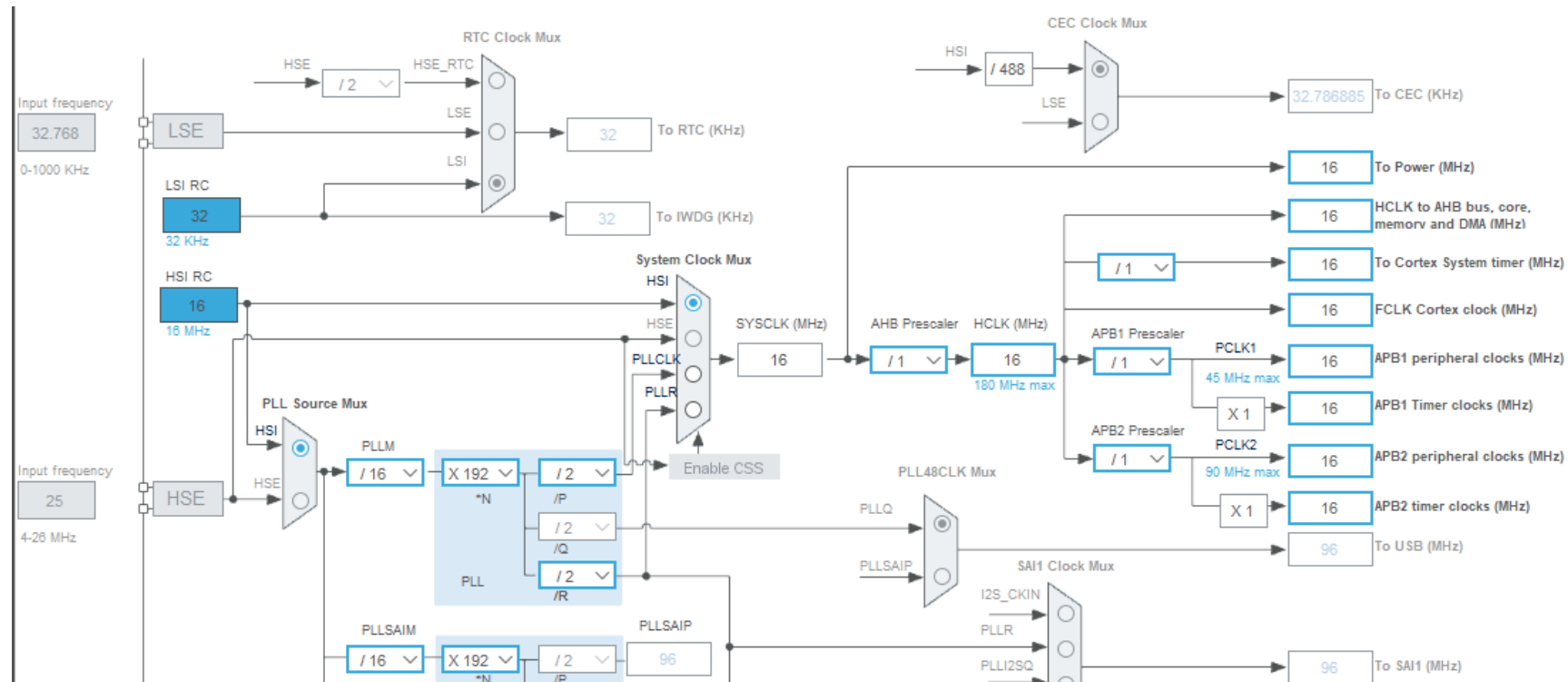
1.4.1 STM32CubeMX.

- En **System Core -> GPIO**, se ajustan algunos parámetros físicos.
- Inputs: las entradas suelen necesitar Pull-up o Pull-down.
- Output se utiliza Level= Low para que el sistema arranque apagado.



1.4.1 STM32CubeMX.

- En Clock Configuration (Reloj): se configura el HCLK (AHB - Advanced High-performance Bus), el cual no debe superar los 180 MHz.

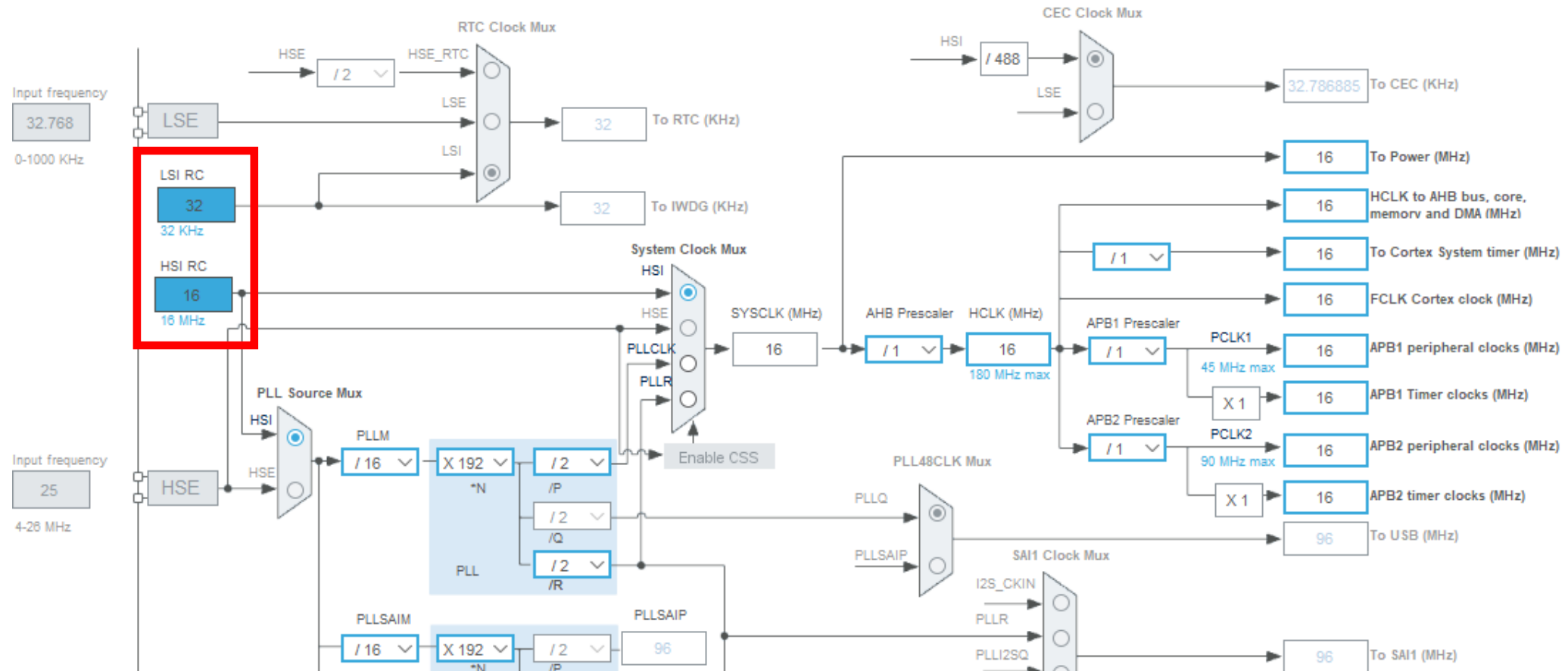


1.4.2 Cube MX. Reloj.

- Dos fuentes de Reloj Internas (RC Oscillators).
- **HSI (High Speed Internal) de 16 MHz:** Es la frecuencia que por defecto utiliza la CPU al arrancar el sistema (precisión de un 1% a temperatura ambiente).
- **LSI (Low Speed Internal) de 32 kHz:** Un oscilador de baja frecuencia que utiliza el watchdog timer.

1.4.2 Cube MX. Reloj.

- Imagen Interna del STM32F446RE en CUBE MX.



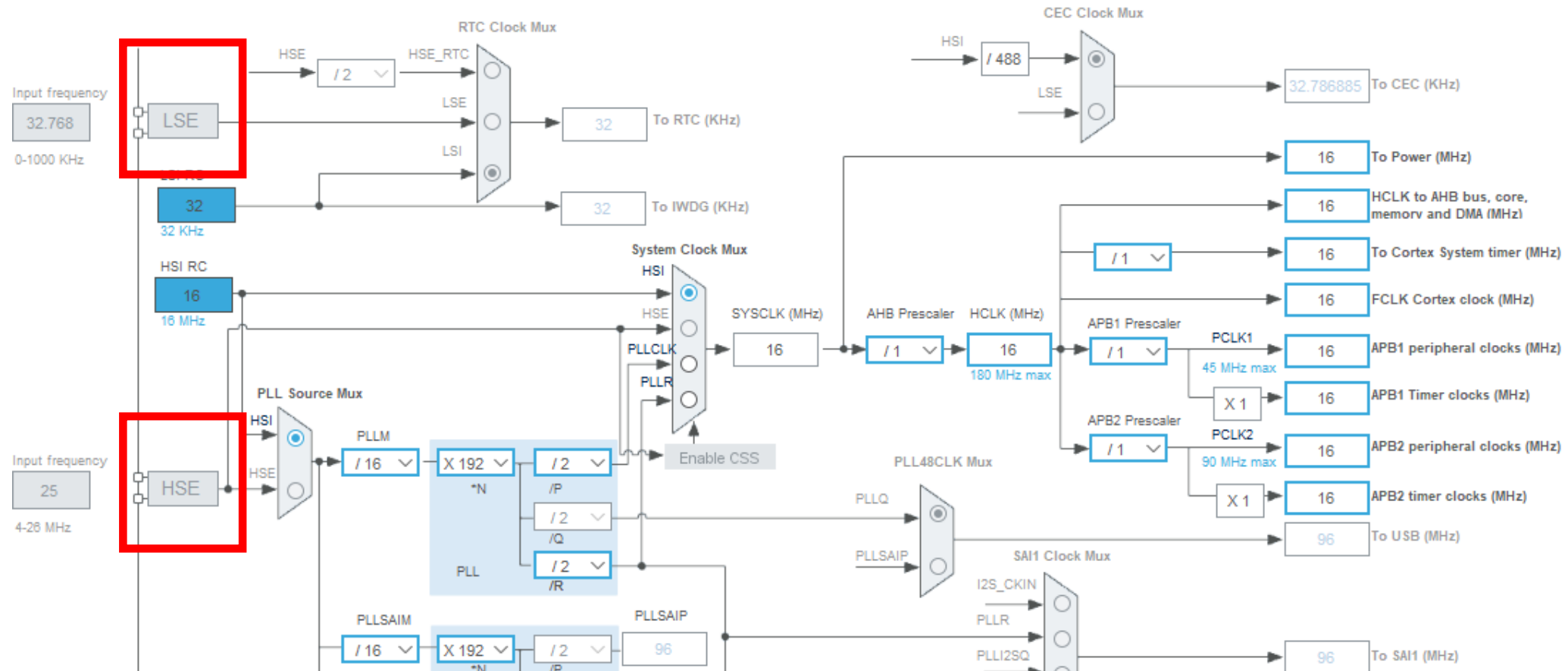
1.4.2 Requisito Crítico: El Reloj (Clock)

Fuentes de Reloj Externas (Crystal/Oscillators):

- **HSE (High Speed External):** Soporta entre 4 y 26 MHz, en la Núcleo se encuentra como X3 y se obtiene una señal más estable.
- **LSE (Low Speed External):** Es un cristal externo de 32.768 kHz (componente X2 en la Núcleo), su función principal es gestionar el RTC (Real-Time Clock).

1.4.2 Requisito Crítico: El Reloj (Clock)

- Imagen Interna del STM32F446RE en CUBE MX.



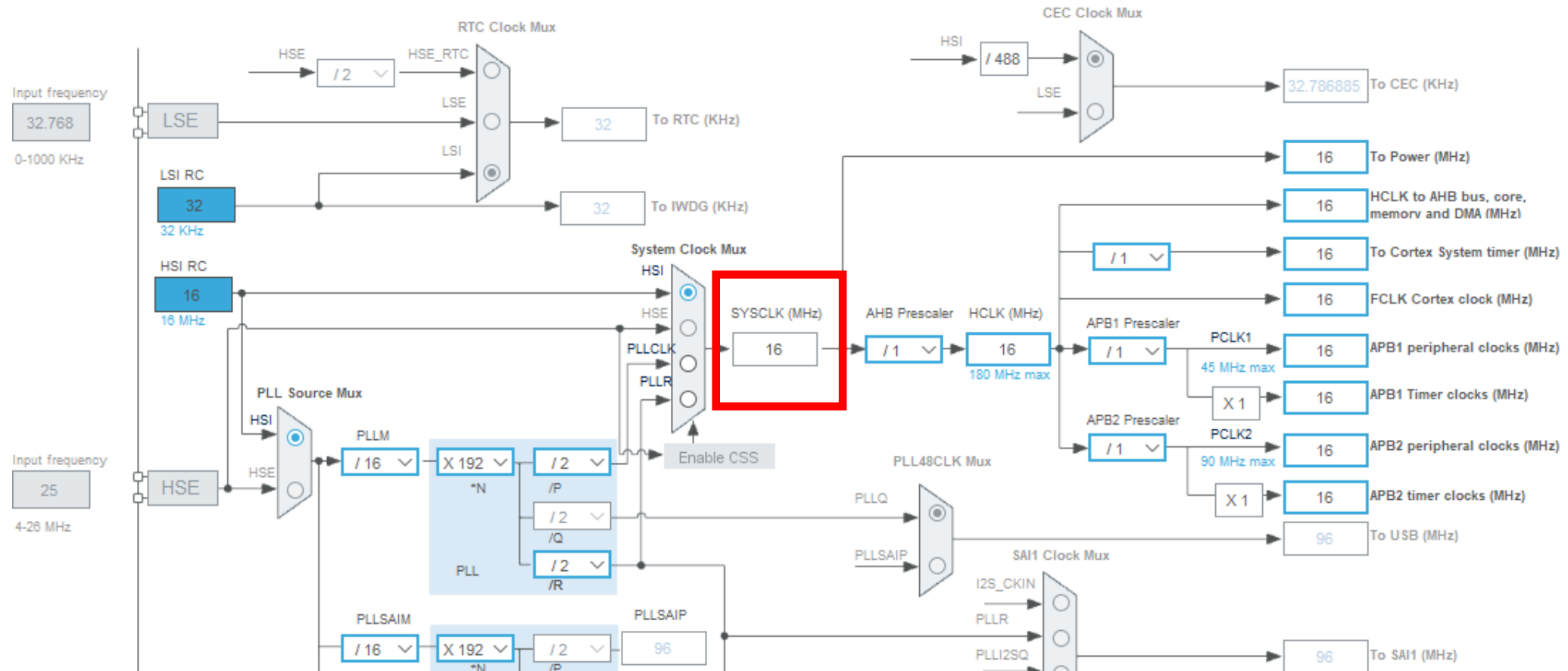
1.4.2 Requisito Crítico: El Reloj (Clock)

El Sistema de Reloj (SYSCLK) y el PLL

- El micro no está limitado a la velocidad de sus osciladores básicos, el SYSCLK (reloj del sistema) puede ser alimentado por el HSI, el HSE o el PLL.
- El PLL permite multiplicar la frecuencia de las fuentes internas o externas para que la CPU alcance su máximo rendimiento de 180 MHz.

1.4.2 Requisito Crítico: El Reloj (Clock)

- Imagen Interna del STM32F446RE en CUBE MX.



1.4.2 Requisito Crítico: El Reloj (Clock)

El reloj principal se distribuye a través de varios buses pasando por los prescalers a:

Buses AHB: Funcionan hasta 180 MHz y alimentan a la CPU y la memoria (nuestras entradas o salidas aquí están conectadas).

Bus APB2 (Alta velocidad): Soporta hasta 90 MHz para periféricos rápidos.

Bus APB1 (Baja velocidad): Limitado a 45 MHz.

Primera Prueba

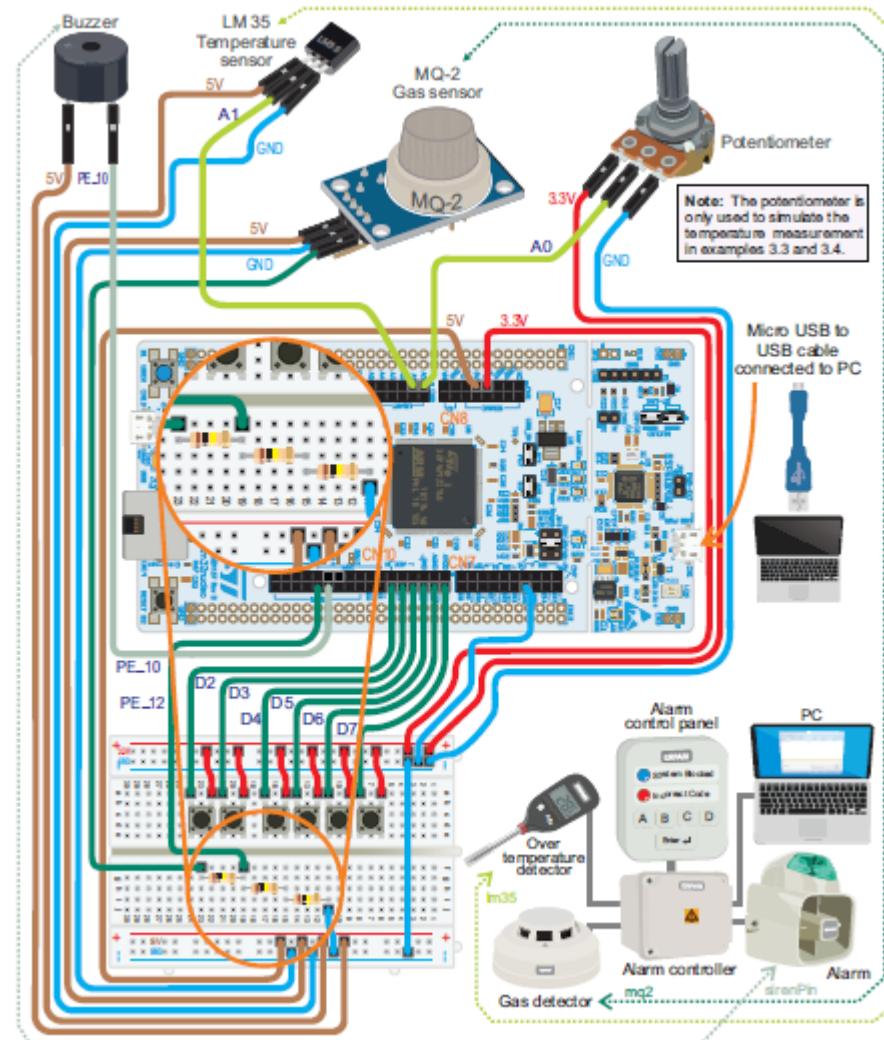
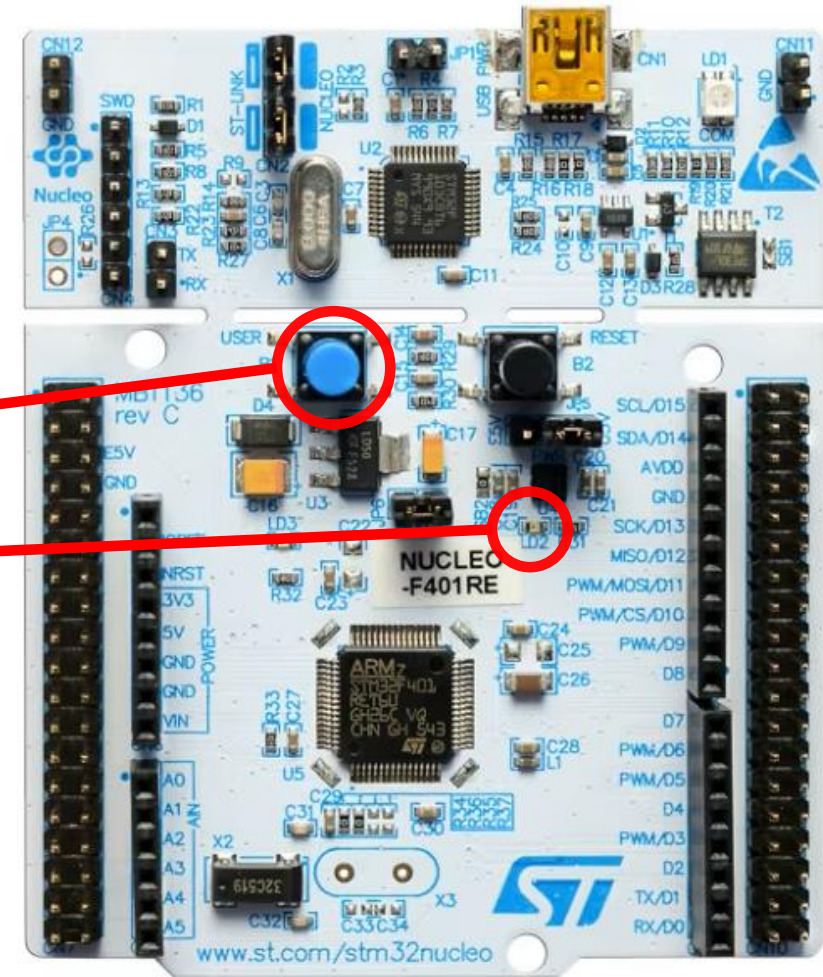


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

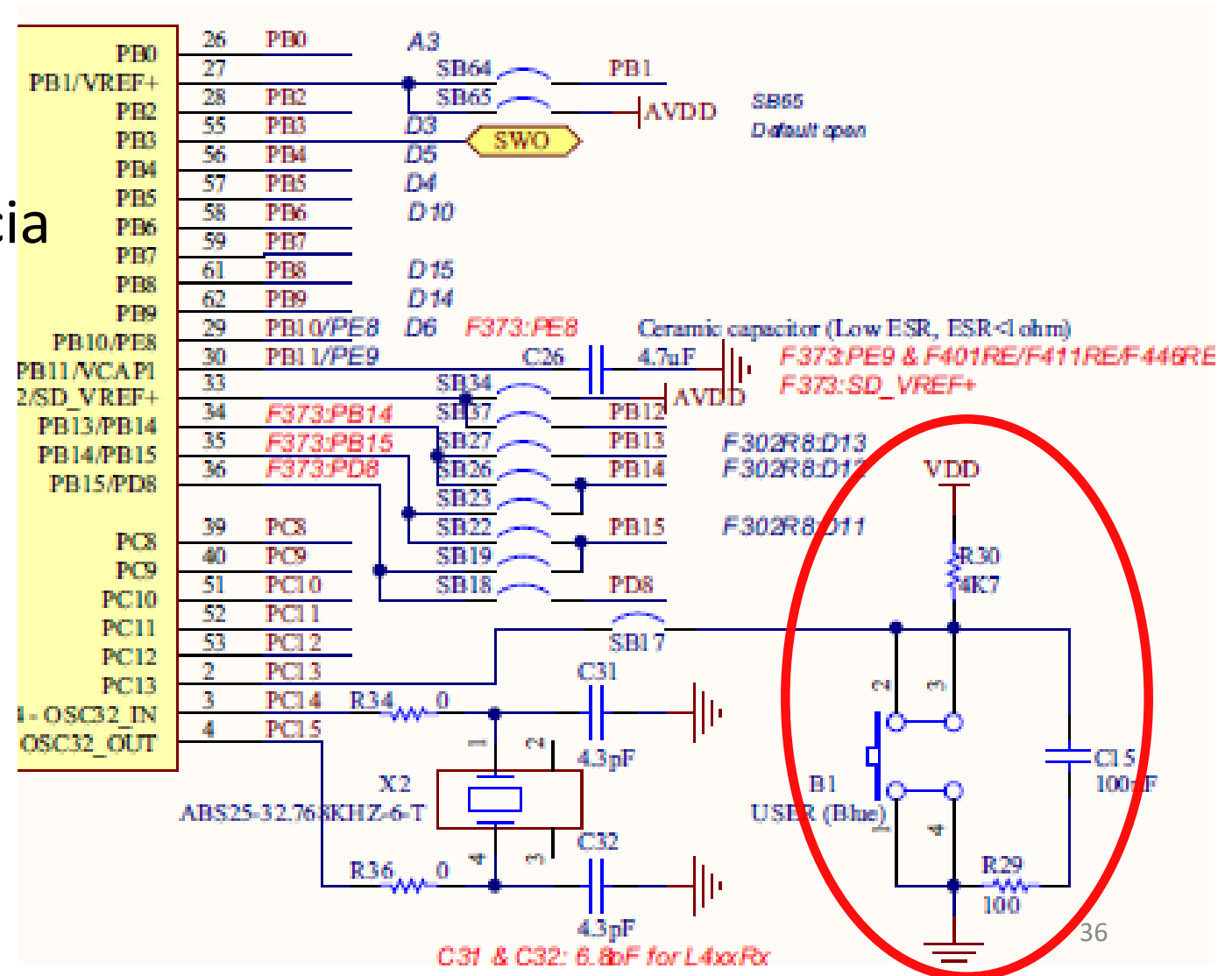
1.5 Primera Prueba.

- No vamos a utilizar para los problemas del libro de la primera semana, el botón de usuario y el led de la Núcleo.
- El *Botón Azul de Usuario* y el *LED*.
- Para los problemas utilizaremos botones y led externos a la placa, en protoboard.



1.5 Primera Prueba.

- El botón de usuario esta conectada a una resistencia Pull Up.



1.5 Primera Prueba.

- Example 0: Configura el botón de usuario de la Nucleo (B1) como entrada dos pines y el led LD2 como salida. Si en la entrada tenemos el valor lógico 1, la salida es en LD2 es 1, caso contrario 0.
- Dos puntos a tener en cuenta resolver el problema.
 - a. Programación con HAL.
 - b. Características del if.

1.5.1 HAL.

- Capa de software desarrollada por STMicroelectronics que actúa como un traductor para:
- Abstracción: no es necesario la dirección de los registros de los puerto A para programar.
- Portabilidad: diferentes familias de microcontroladores.

1.5.1 HAL_GPIO_ReadPin

- Lee el estado lógico (0 o 1) en un pin configurado como entrada.
- `HAL_GPIO_ReadPin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);`
- `GPIOx`: El puerto donde está el pin (ej: `GPIOA`, `GPIOB`).
- `GPIO_Pin`: El número del pin específico (ej: `GPIO_PIN_5`).
- Devuelve un valor de tipo enumerado:
- `GPIO_PIN_SET` (que equivale a un 1 lógico).
- `GPIO_PIN_RESET` (que equivale a un 0 lógico).

1.5.1 HAL_GPIO_WritePin

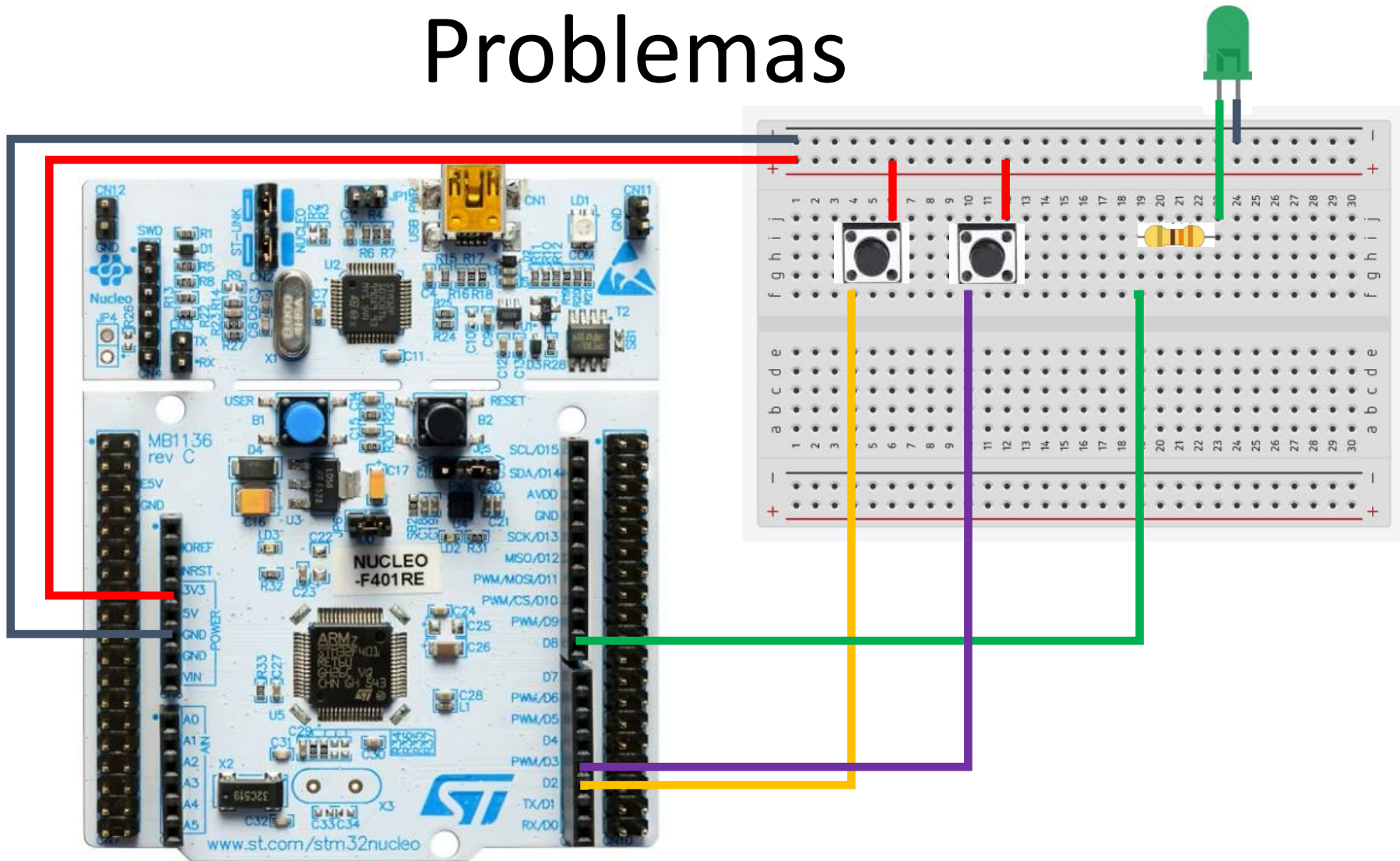
- Escribe un estado lógico (0 o 1) en un pin configurado como salida.
- `HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState);`
- `GPIOx`: El puerto (ej: `GPIOA`).
- `GPIO_Pin`: El pin (ej: `GPIO_PIN_5`).
- `PinState`: El estado que quieres aplicar, `GPIO_PIN_SET` (encender) o `GPIO_PIN_RESET` (apagar).

1.5.2 Primera Programa.

```
#include "main.h"
#include "gpio.h"
void SystemClock_Config(void);
int main(void) {

    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    while (1){
        if( HAL_GPIO_ReadPin(boton1_GPIO_Port, boton1_Pin) ){
            HAL_GPIO_WritePin(led1_GPIO_Port, led1_Pin,GPIO_PIN_SET);
        }else{
            HAL_GPIO_WritePin(led1_GPIO_Port, led1_Pin,GPIO_PIN_RESET);
        }
    }
}
```

Problemas



1.6 Problemas.

Iniciamos con los problemas del Capítulo 1:

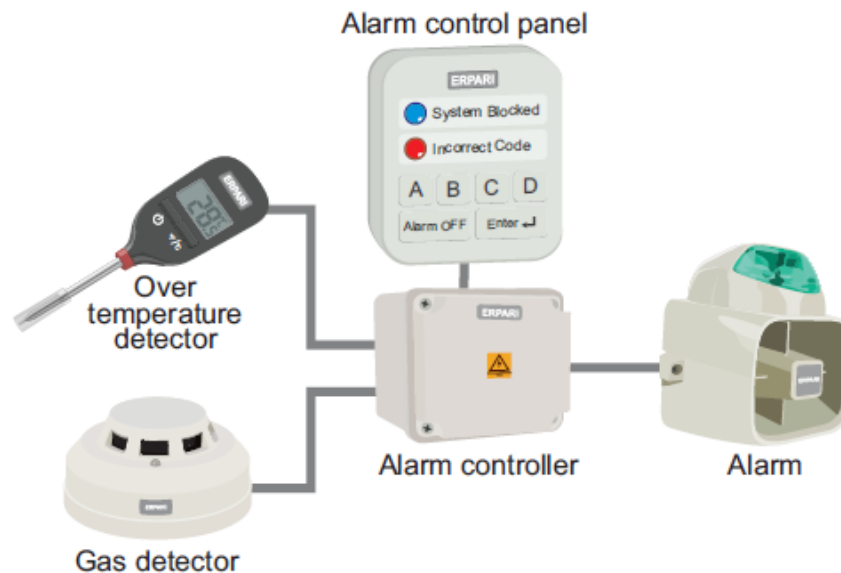


Figure 1.3 The smart home system that is implemented in this chapter.

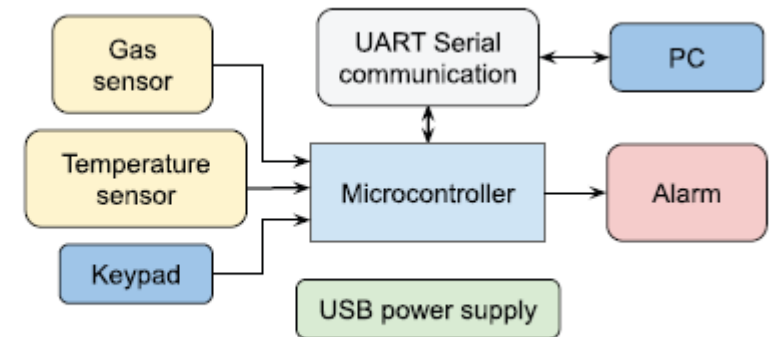
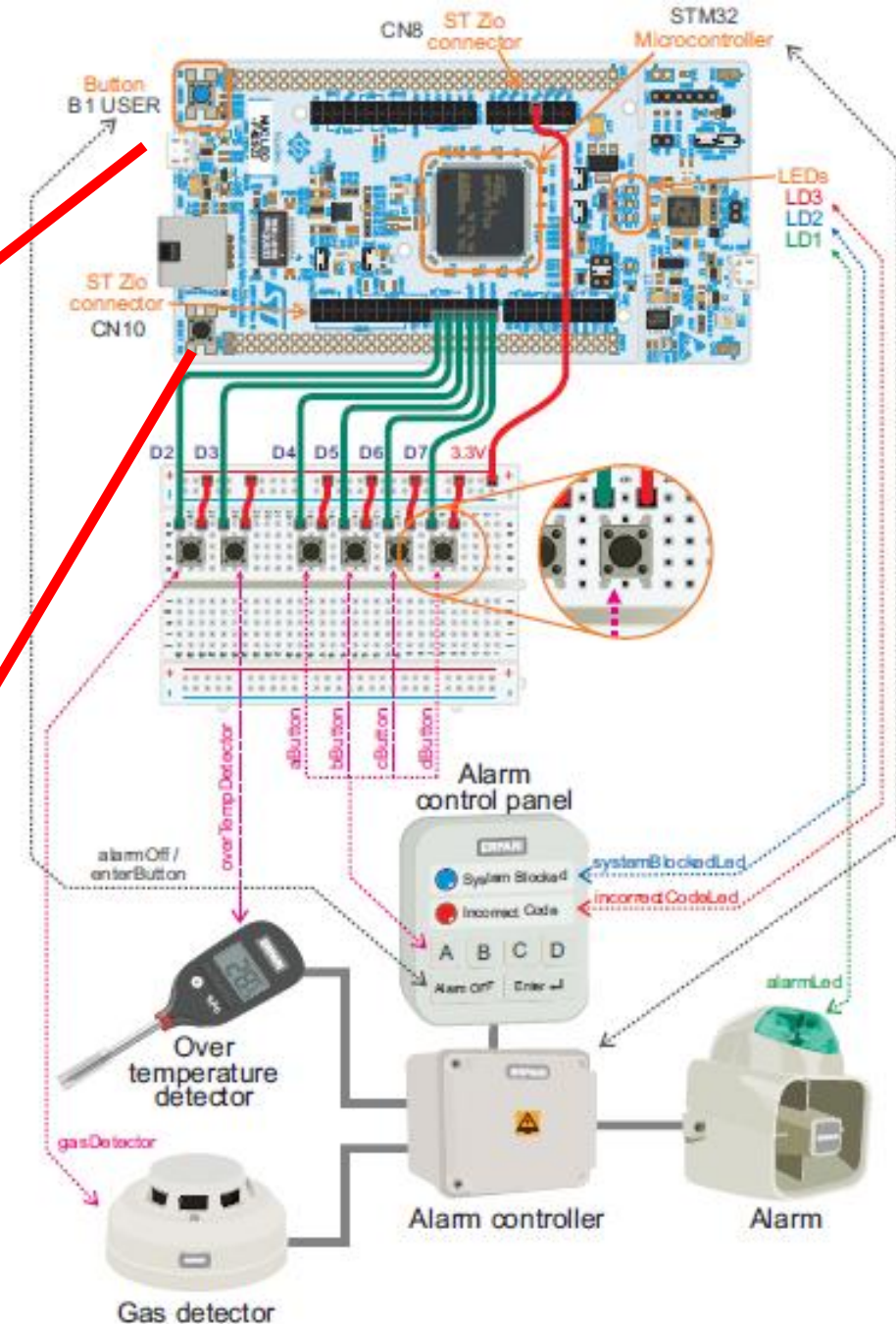
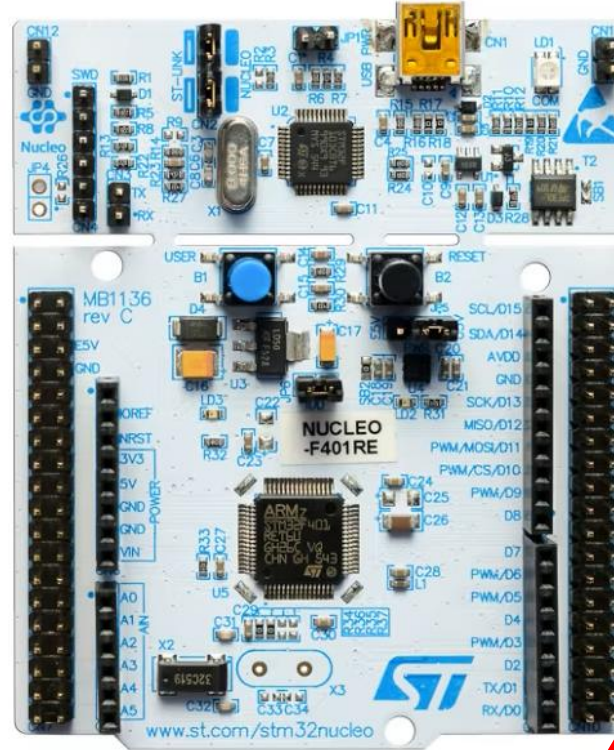


Figure 1.2 Diagram of the proposed smart home system.

Extractos del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers (pag.5 y 6)
<https://www.arm.com/resources/education/books/designing-embedded-systems>

1.6 Problemas.

Cambiaremos la placa de desarrollo:



1.6 Problemas.

Iniciamos con los problemas del Capítulo 1:

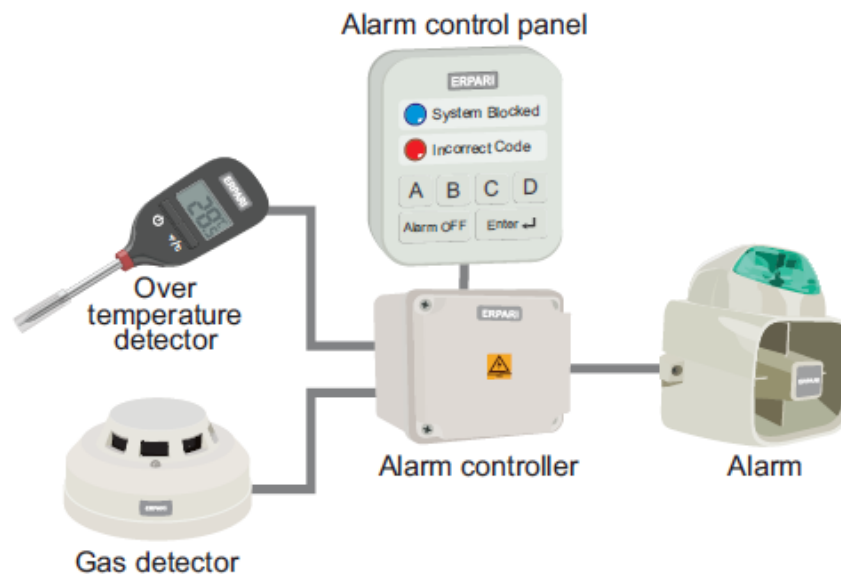


Figure 1.3 The smart home system that is implemented in this chapter.

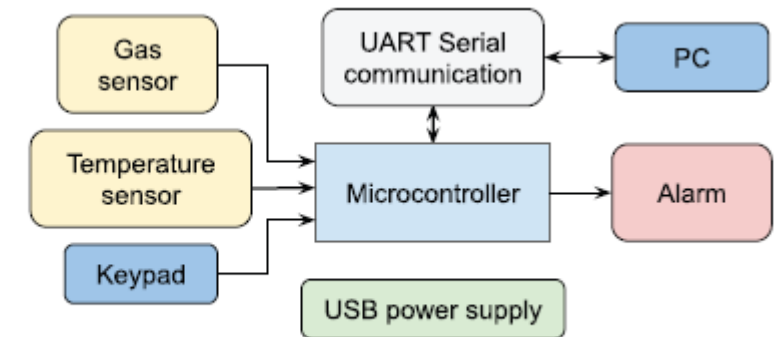
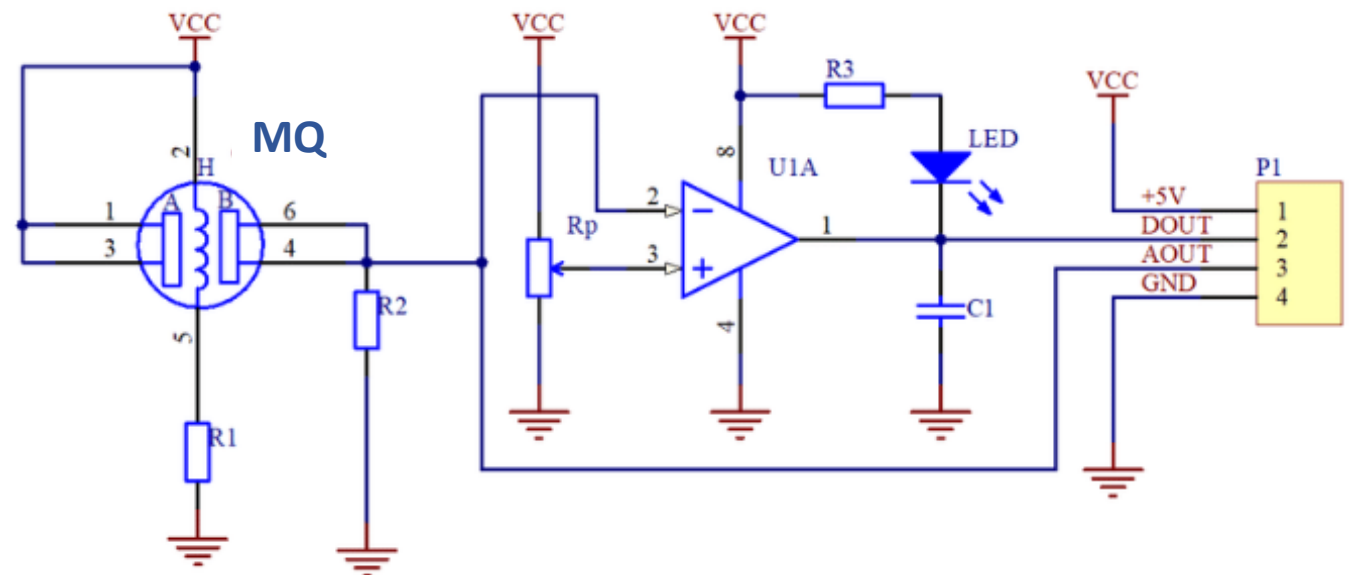


Figure 1.2 Diagram of the proposed smart home system.

Extractos del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers (pag.5 y 6)
<https://www.arm.com/resources/education/books/designing-embedded-systems>

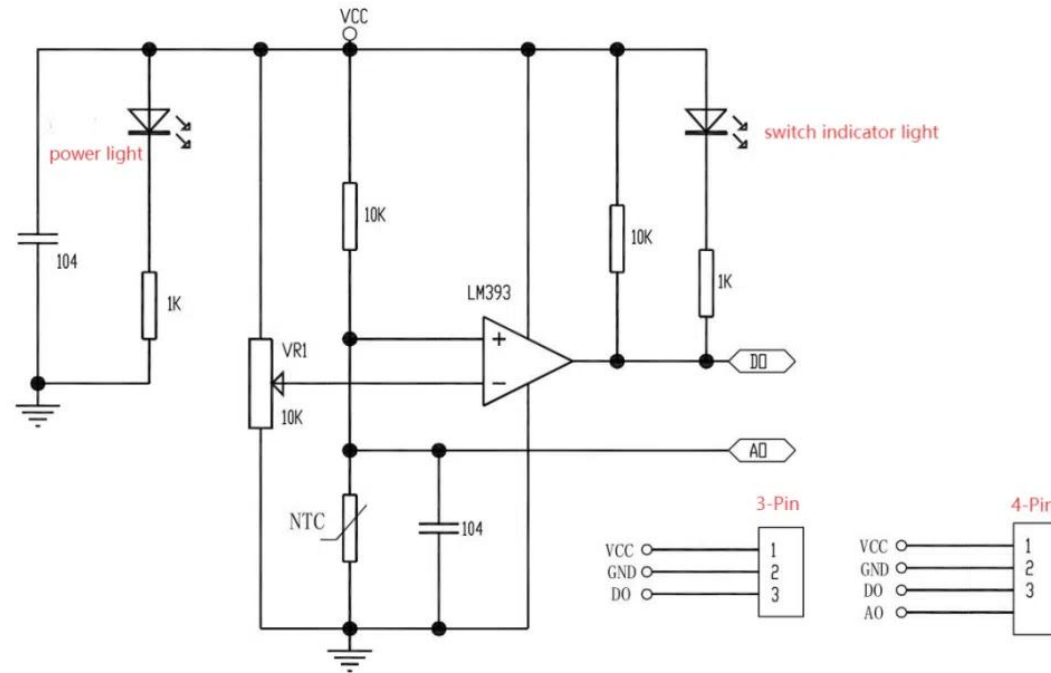
1.6 Problemas.

- MQ 2: sensor diseñado para detectar concentraciones de GLP, propano, metano, alcohol, hidrógeno y humo en el aire.
- La placa tiene un comparador cuya salida esta en función de la magnitud del sensor y una tensión que trabaja como un Set-Point.



1.6 Problemas.

- EM0603: placa con un termistor diseñada para medir temperatura.
- La placa tiene un comparador cuya salida esta en función de la magnitud del sensor y una tensión que trabaja SP.



1.6.1 Example 1.1.

- En el problema se definen dos pines, uno como entrada y otra como salida. La entrada representa la salida digital de un sensor de gas (como el MQ2) y la salida representa la activación de una alarma.
- En la figura se explica la lógica del problema.

Lines in Code 1.1	New code to be used
15 if (gasDetector == ON) { 16 alarmLed = ON; 17 } 18 19 if (gasDetector == OFF) { 20 alarmLed = OFF; 21 }	16 if (gasDetector == ON) { 17 alarmLed = ON; 18 } else { 19 alarmLed = OFF; 20 } 21

<https://www.arm.com/resources/education/books/designing-embedded-systems>

```
1 #include "mbed.h"  
2 #include "arm_book_lib.h"
```

Include the libraries "mbed.h" and "arm_book_lib.h"

```
3  
4 int main()  
5 {  
6     DigitalIn gasDetector(D2);  
7  
8     DigitalOut alarmLed(LED1);  
9  
10    gasDetector.mode(PullDown);  
11  
12    alarmLed = OFF;  
13  
14    while ( true ) {  
15        if ( gasDetector == ON ) {  
16            alarmLed = ON;  
17        }  
18  
19        if ( gasDetector == OFF ) {  
20            alarmLed = OFF;  
21        }  
22    }  
23 }
```

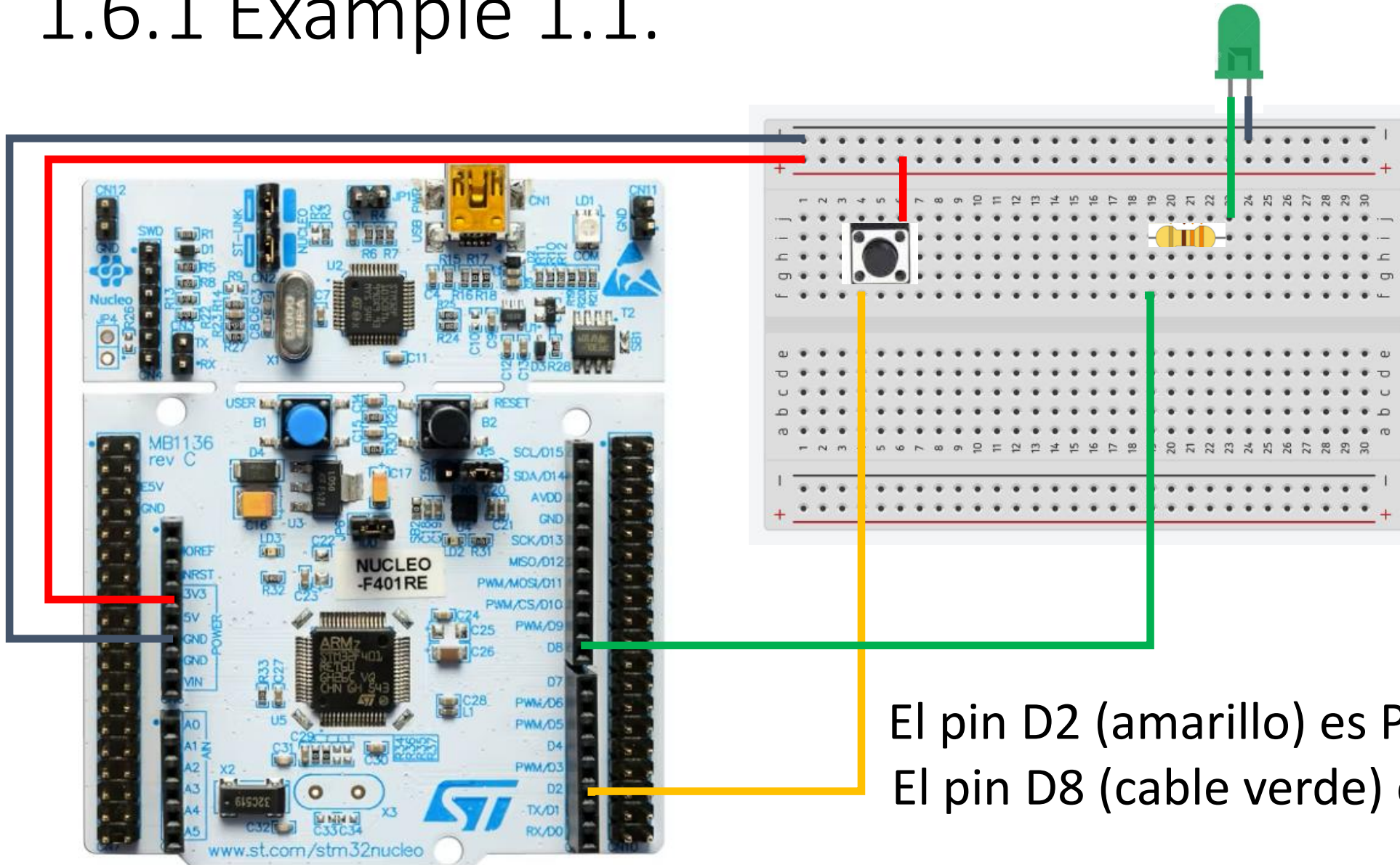
- A digital input object (DigitalIn) named *gasDetector* is declared and assigned to D2.
- A digital output object (DigitalOut) named *alarmLed* is declared and assigned to LED1.
- *gasDetector* is configured with an internal pull-down resistor.
- *alarmLed* (LED1) is assigned OFF.

Do forever loop:

- If *gasDetector* is active, *alarmLed* is assigned ON.

- If *gasDetector* is not active, *alarmLed* is assigned OFF.

1.6.1 Example 1.1.



1.6.1 Example 1.1.

```
#include "main.h"
#include "gpio.h"
```

```
void SystemClock_Config(void);
int main(void) {
```

```
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    HAL_GPIO_WritePin(alarmLed_GPIO_Port, alarmLed_Pin, GPIO_PIN_RESET);
    HAL_Delay(200);
    while (1) {
        if (HAL_GPIO_ReadPin(gasDetector_GPIO_Port, gasDetector_Pin) == 1) {
            HAL_GPIO_WritePin(alarmLed_GPIO_Port, alarmLed_Pin, GPIO_PIN_SET);
        }
        if (HAL_GPIO_ReadPin(gasDetector_GPIO_Port, gasDetector_Pin) == 0) {
            HAL_GPIO_WritePin(alarmLed_GPIO_Port, alarmLed_Pin, GPIO_PIN_RESET);
        }
    }
}
```

```
#include "mbed.h"
#include "arm_book_lib.h"
```

Include the libraries "mbed.h" and "arm_book_lib.h"

```
int main()
{
    DigitalIn gasDetector(D2);
    DigitalOut alarmLed(LED1);
    gasDetector.mode(PullDown);
    alarmLed = OFF;

    while ( true ) {
        if ( gasDetector == ON ) {
            alarmLed = ON;
        }

        if ( gasDetector == OFF ) {
            alarmLed = OFF;
        }
    }
}
```

- A digital input object (DigitalIn) named *gasDetector* is declared and assigned to D2.
- A digital output object (DigitalOut) named *alarmLed* is declared and assigned to LED1.
- *gasDetector* is configured with an internal pull-down resistor.
- *alarmLed* (LED1) is assigned OFF.

Do forever loop:

- If *gasDetector* is active, *alarmLed* is assigned ON.
- If *gasDetector* is not active, *alarmLed* is assigned OFF.

1.6.2 Example 1.2.

- En el problema se definen tres pines, dos como entrada y uno como salida. Las entradas representan la salida digital un sensor de gas y otro de temperatura. La salida representa la activación de una alarma.
- En la figura se explica la lógica del problema.

```
1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8
9     DigitalOut alarmLed(LED1);
10
11     gasDetector.mode(PullDown);
12     overTempDetector.mode(PullDown);
13
14     while ( true ) {
15
16         if ( gasDetector || overTempDetector ) {
17             alarmLed = ON;
18         }
19         alarmLed = OFF;
20     }
21 }
22 }
```

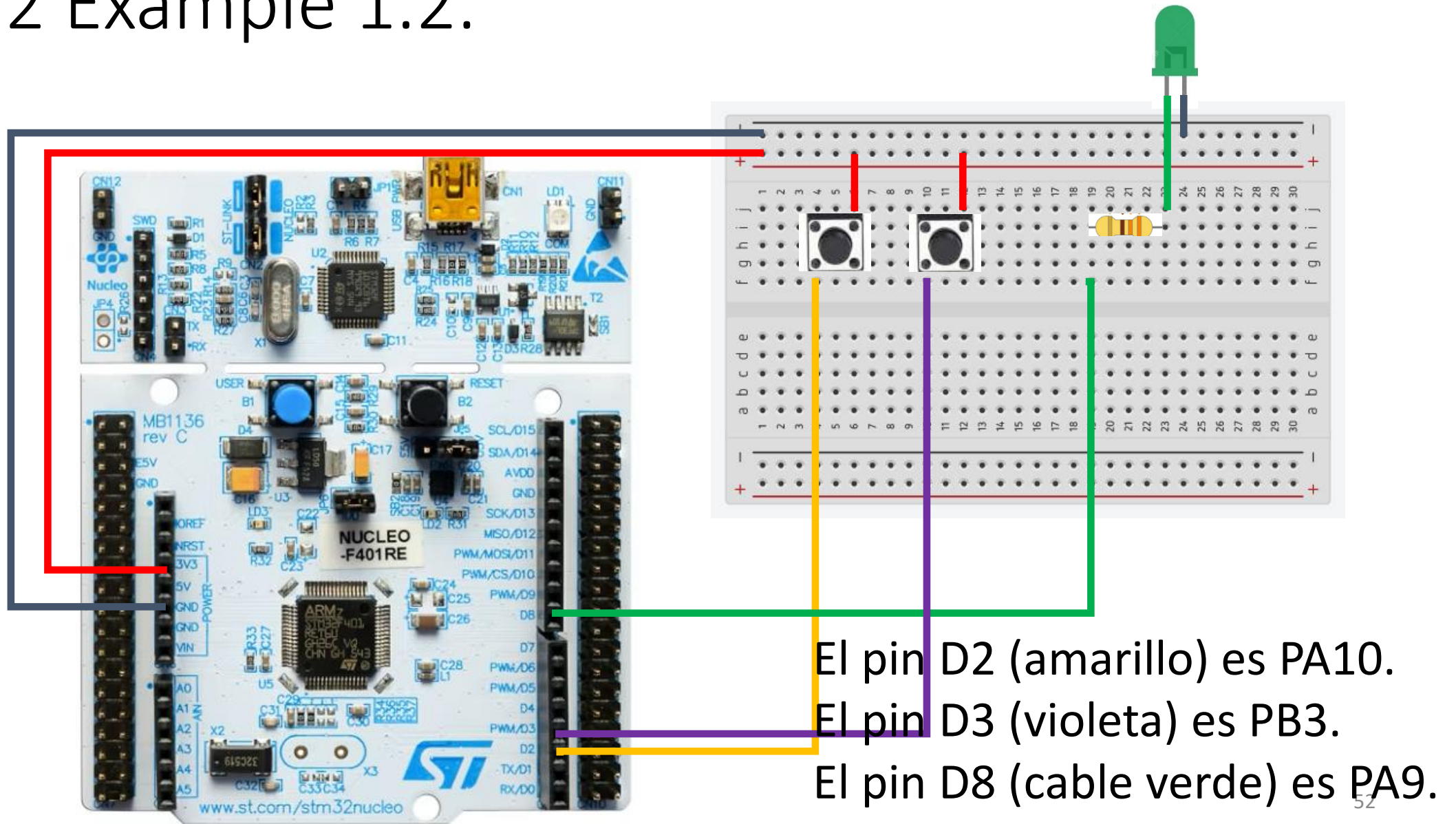
Include the libraries "mbed.h" and "arm_book_lib.h"

- Digital input objects named *gasDetector* and *overTempDetector* are declared and assigned to D2 and D3, respectively.
- Digital output object named *alarmLed* is declared and assigned to LED1.
- *gasDetector* and *overTempDetector* are configured with internal pull-down resistors.

Do forever loop:

- If *gasDetector* or *overTempDetector* are active, *alarmLed* is assigned ON.
- Else, *alarmLed* is assigned OFF.

1.6.2 Example 1.2.



1.6.2 Example 1.2.

```
#include "main.h"
#include "gpio.h"

void SystemClock_Config(void);
int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    HAL_GPIO_WritePin(alarmLed_GPIO_Port, alarmLed_Pin, GPIO_PIN_RESET);
    HAL_Delay(200);
    while (1){
        if(HAL_GPIO_ReadPin(gasDetector_GPIO_Port, gasDetector_Pin) || HAL_GPIO_ReadPin(overTempDetector_GPIO_Port, overTempDetector_Pin)){
            HAL_GPIO_WritePin(alarmLed_GPIO_Port, alarmLed_Pin, GPIO_PIN_SET);
        }else{
            HAL_GPIO_WritePin(alarmLed_GPIO_Port, alarmLed_Pin, GPIO_PIN_RESET);
        }
    }
}
```

```
1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8
9     DigitalOut alarmLed(LED1);
10
11     gasDetector.mode(PullDown);
12     overTempDetector.mode(PullDown);
13
14     while ( true ) {
15
16         if ( gasDetector || overTempDetector ) {
17             alarmLed = ON;
18         }
19         alarmLed = OFF;
20     }
21 }
22 }
```

Include the libraries "mbed.h" and "arm_book_lib.h"

- Digital input objects named *gasDetector* and *overTempDetector* are declared and assigned to D2 and D3, respectively.
- Digital output object named *alarmLed* is declared and assigned to LED1.
- *gasDetector* and *overTempDetector* are configured with internal pull-down resistors.

Do forever loop:

- If *gasDetector* or *overTempDetector* are active, *alarmLed* is assigned ON.
- Else, *alarmLed* is assigned OFF.

1.6.3 Example 1.3.

- En el problema se definen cuatro pines, tres como entrada y uno como salida. Las entradas representan la salida digital un sensor de gas y otro de temperatura.

La salida representa la activación de una alarma.

El botón reset apaga la alarma si se acciono.

- En la figura se explica la lógica del problema.

```
1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8     DigitalIn alarmOffButton(BUTTON1);
9
10    DigitalOut alarmLed(LED1);
11
12    gasDetector.mode(PullDown);
13    overTempDetector.mode(PullDown);
14
15    alarmLed = OFF;
16
17    bool alarmState = OFF;
18
19    while ( true ) {
20
21        if ( gasDetector || overTempDetector ) {
22            alarmState = ON;
23        }
24
25        alarmLed = alarmState;
26
27        if ( alarmOffButton ) {
28            alarmState = OFF;
29        }
30    }
31 }
```

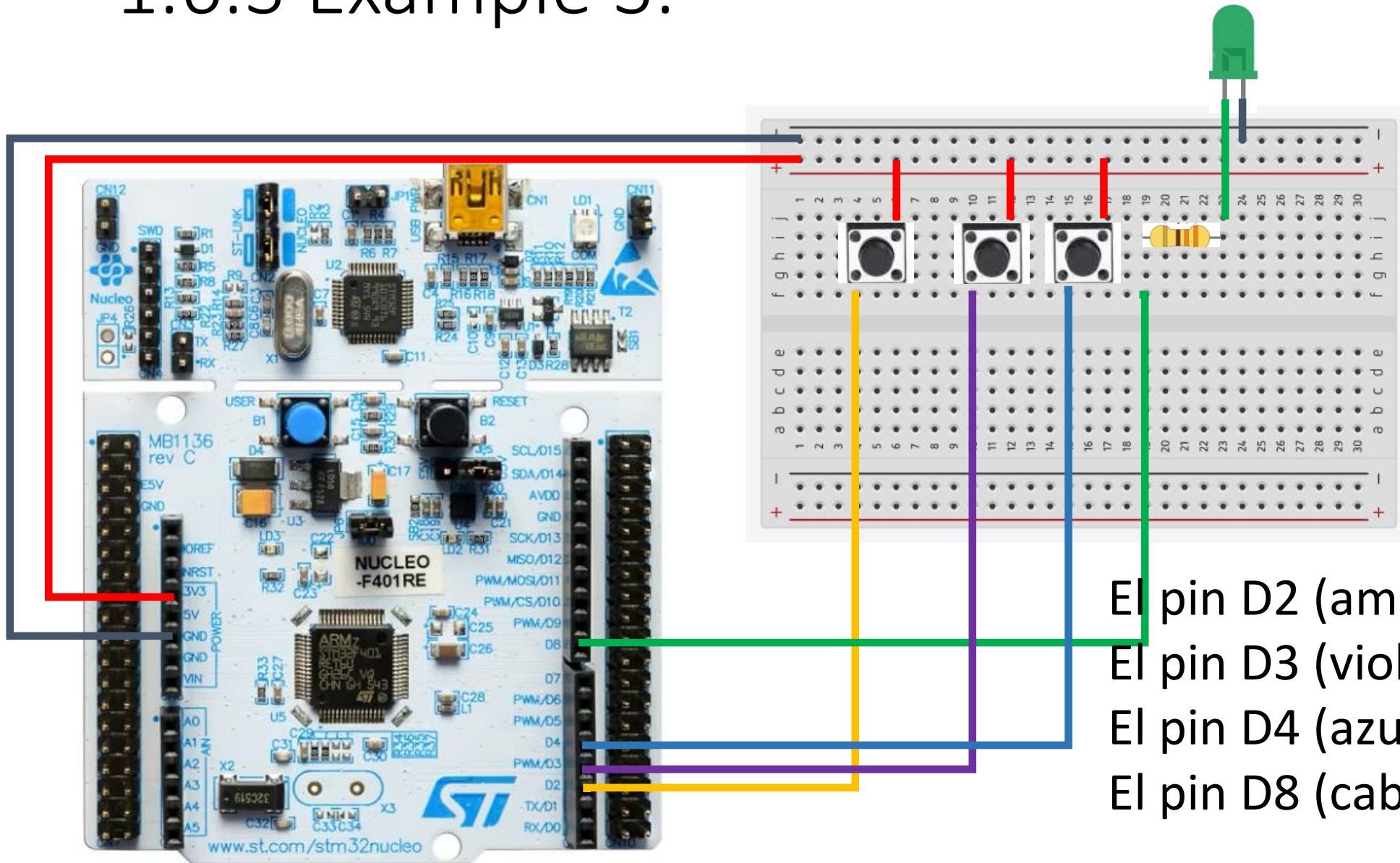
Include the libraries "mbed.h" and "arm_book_lib.h"

- Digital input objects named *gasDetector*, *overTempDetector* and *alarmOffButton*, are declared and assigned to D2, D3, and BUTTON1 (B1 USER), respectively.
- Digital output object named *alarmLed* is declared and assigned to LED1.
- *gasDetector* and *overTempDetector* are configured with internal pull-down resistors.
- *alarmLed* is assigned OFF.
- *alarmState* is declared and assigned OFF.

Do forever loop:

- If *gasDetector* or *overTempDetector* are active, assign *alarmState* with ON.
- Assign *alarmLed* with *alarmState*.
- If *alarmOffButton*, assign *alarmState* with OFF.

1.6.3 Example 3.



El pin D2 (amarillo) es PA10.

El pin D3 (violeta) es PB3.

El pin D4 (azul) es PB5.

El pin D8 (cable verde) es PA9.

1.6.3 Example 1.3.

```
#include <stdbool.h>
#include "main.h"
#include "gpio.h"
void SystemClock_Config(void);
int main(void) {
    bool alarmState=0;
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    HAL_GPIO_WritePin(led_GPIO_Port, led_Pin, alarmState);
    HAL_Delay(100);
    while (1){
        if(HAL_GPIO_ReadPin(sensor_gas_GPIO_Port,sensor_gas_Pin)||HAL_GPIO_ReadPin(sensor_
temp_GPIO_Port,sensor_temp_Pin)){
            alarmState=1;
        }
        HAL_GPIO_WritePin(led_GPIO_Port, led_Pin, alarmState);
        if(HAL_GPIO_ReadPin(boton_GPIO_Port, boton_Pin)){
            alarmState=0;
        }
    }
}
```

```
1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8     DigitalIn alarmOffButton(BUTTON1);
9
10    DigitalOut alarmLed(LED1);
11
12    gasDetector.mode(PullDown);
13    overTempDetector.mode(PullDown);
14
15    alarmLed = OFF;
16
17    bool alarmState = OFF;
18
19    while ( true ) {
20
21        if ( gasDetector || overTempDetector ) {
22            alarmState = ON;
23        }
24
25        alarmLed = alarmState;
26
27        if ( alarmOffButton ) {
28            alarmState = OFF;
29        }
30    }
31 }
```

Include the libraries "mbed.h"
and "arm_book_lib.h"

- Digital input objects named *gasDetector*, *overTempDetector* and *alarmOffButton*, are declared and assigned to D2, D3, and BUTTON1 (B1 USER), respectively.
- Digital output object named *alarmLed* is declared and assigned to LED1.
- *gasDetector* and *overTempDetector* are configured with internal pull-down resistors.
- *alarmLed* is assigned OFF.
- *alarmState* is declared and assigned OFF.

Do forever loop:

- If *gasDetector* or *overTempDetector* are active, assign *alarmState* with ON.
- Assign *alarmLed* with *alarmState*.
- If *alarmOffButton*, assign *alarmState* with OFF.

Bibliografia.

Lutenberg A., Gomez P. and Pernia E., 2022. A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Arm Education Media.

Ortiz J.P., Malagón P. J., Cárdenas R. and Gómez J.J., 2025. Fundamentos teóricos de sistemas basados en microcontrolador STM32, Sistemas Digitales II Sistemas Electrónicos, Universidad Politécnica de Madrid.

Cem Ünsalan C., Yücel M. E. and Gürhan H. D., 2018. Digital Signal Processing using Arm Cortex-M based Microcontrollers-Theory and Practice, Arm Education Media.

YIU J., 2019. System-on-Chip Design with Arm® Cortex® -M Processors, Arm Education Media